

## MODULE 6

# Streams and Functional Programming

Leverage Java Streams and functional programming to write concise, expressive, and efficient code.



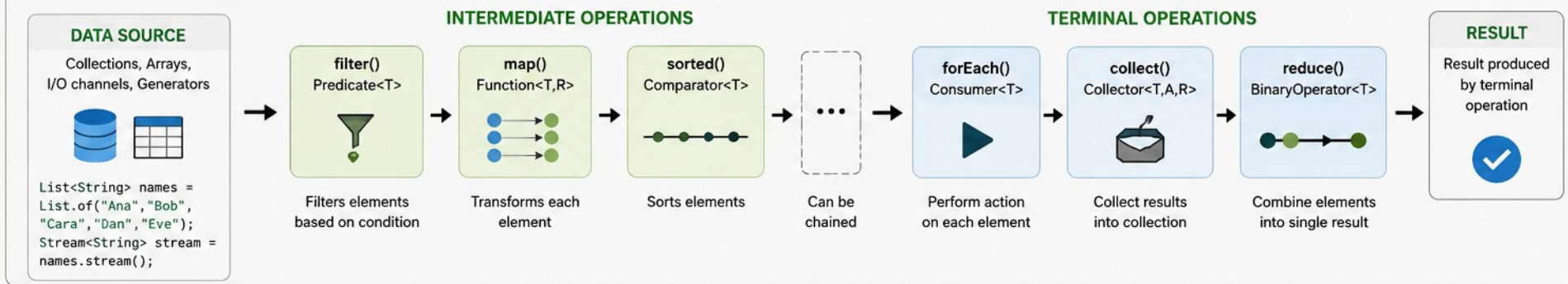




# STREAMS AND FUNCTIONAL PROGRAMMING

Declarative, Functional Style for Processing Collections

## 1. STREAM PIPELINE



## 2. FUNCTIONAL INTERFACES (CORE BUILDING BLOCKS)



## 3. EXAMPLE: PROCESSING WITH STREAMS

### INPUT

List<Integer> numbers  
[ 5, 2, 9, 1, 5, 6 ]

```
int sumOfSquares = numbers.stream()
    .filter(n -> n > 2) // 5, 9, 5, 6
    .map(n -> n * n) // 25, 81, 25, 36
    .distinct() // 25, 81, 36
    .sorted() // 25, 36, 81
    .reduce(0, Integer::sum); // 142
```

### OUTPUT

**142**

(Sum of squares)

## 4. IMPERATIVE vs FUNCTIONAL PARADIGM

### IMPERATIVE STYLE (How to do it)

```
int sum = 0;
for (int n : numbers) {
    if (n > 2) {
        int sq = n * n;
        sum += sq;
    }
}
```

VS

### FUNCTIONAL STYLE (What to do)

```
int sum = numbers.stream()
    .filter(n -> n > 2)
    .map(n -> n * n)
    .reduce(0, Integer::sum);
```



Functional style is more concise, readable, and easy to reason about. Focus on WHAT should be done, not HOW.

## 5. KEY CHARACTERISTICS



### Declarative

Express logic without controlling flow



### Composable

Operations can be chained seamlessly



### Lazy Evaluation

Intermediate operations are executed only when needed



### Efficient

Optimized for performance, can leverage parallelism

## 6. BENEFITS



### Concise & Readable

Less boilerplate code, intent is clear



### Easier Maintenance

Modular and composable operations



### Parallel Processing

Easy to parallelize with `parallelStream()`



### Better Performance

Internal optimizations by Stream API

★ **NOTE:** Streams do not change the original data source. They provide a functional view for processing data.

Common Data Sources:

Collection Array Files Generate Iterate

# MODULE OVERVIEW

*Leverage Java Streams and functional programming features to write concise, expressive, and efficient code.*

## WHAT YOU WILL LEARN

- Introduction to Streams and the Stream Pipeline
- Creating Streams from Collections, Arrays, and other sources
- Intermediate Operations: filter, map, flatMap, sorted, distinct, peek, limit, skip
- Terminal Operations: forEach, collect, reduce, count, anyMatch, allMatch, noneMatch, findFirst
- Stream Pipeline Concepts: lazy evaluation, short-circuiting, stateless vs stateful operations
- Functional Interfaces: Predicate, Function, Consumer, Supplier
- Lambda Expressions and Method References
- Optional class and handling absence of values
- Best Practices and Performance Considerations

**KEY TAKEAWAY** Streams + Functional Programming enable declarative, readable, and efficient data processing while reducing boilerplate.

# FUNCTIONAL PROGRAMMING IN JAVA

*Treat computation as the evaluation of mathematical functions*

## CORE PRINCIPLES



### FUNCTIONS AS FIRST-CLASS CITIZENS

Functions can be assigned, passed and returned.



### IMMUTABILITY

Prefer immutable data and avoid changing state.



### NO SIDE EFFECTS

Functions should not modify external state; pure functions.



### HIGHER-ORDER FUNCTIONS

Functions that take other functions as arguments or return them.



### FUNCTION COMPOSITION

Build complex operations by composing simple functions.

## ENABLING FEATURES IN JAVA



Lambda Expressions



Functional Interfaces



Stream API



Optional Class



Method References

## FROM IMPERATIVE TO FUNCTIONAL STYLE

### Imperative Approach (How)

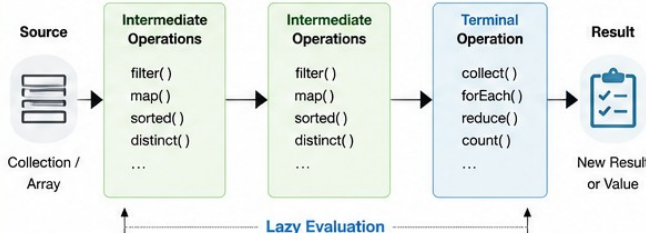
```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenSquares = new ArrayList<>();
for (int n : numbers) {
    if (n % 2 == 0) {
        evenSquares.add(n * n);
    }
}
```

### Functional Approach (What)

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenSquares = numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());
```

Focus on **what** to do, not **how** to do it.

## STREAM PIPELINE WORKFLOW



## BENEFITS OF FUNCTIONAL PROGRAMMING



### CONCISENESS

Less boilerplate code, more expressive.



### BETTER READABILITY

Code intent is clear and easier to understand.



### EASY TESTING

Pure functions are predictable and easy to test.



### PARALLELISM READY

Immutable, stateless code works well with parallel streams.



### COMPOSABILITY

Small functions can be combined to build complex logic.

## FUNCTIONAL INTERFACE EXAMPLES

| Interface     | Method Signature  | Example Use                                                     | Lambda Examples                               |
|---------------|-------------------|-----------------------------------------------------------------|-----------------------------------------------|
| Predicate<T>  | boolean test(T t) | Filtering ( <code>filter()</code> )<br>Validation               | <code>x -&gt; x &gt; 10</code>                |
| Function<T,R> | R apply(T t)      | Mapping ( <code>map()</code> )<br>Transformation                | <code>s -&gt; s.toUpperCase()</code>          |
| Consumer<T>   | void accept(T t)  | Processing ( <code>forEach()</code> )<br>Side-effect operations | <code>s -&gt; System.out.println(s)</code>    |
| Supplier<T>   | T get()           | Providing values<br>( <code>Supply</code> )                     | <code>() -&gt; new ArrayList&lt;&gt;()</code> |



Functional programming leads to cleaner, more maintainable, and scalable Java applications.

**ALSO FROM THIS TOPIC** Less error-prone — immutable data & pure functions reduce bugs • Designed for modern systems — aligns with streams, reactive & cloud apps

# IMPERATIVE VS FUNCTIONAL APPROACH

*The same task, expressed two different ways.*

## TRADITIONAL IMPERATIVE APPROACH

```
List<Integer> result = new ArrayList<>();  
for (int n : numbers) {  
    if (n % 2 == 0) {  
        result.add(n * 2);  
    }  
}
```

## FUNCTIONAL APPROACH WITH STREAMS

```
List<Integer> result = numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .map(n -> n * 2)  
    .collect(Collectors.toList());
```

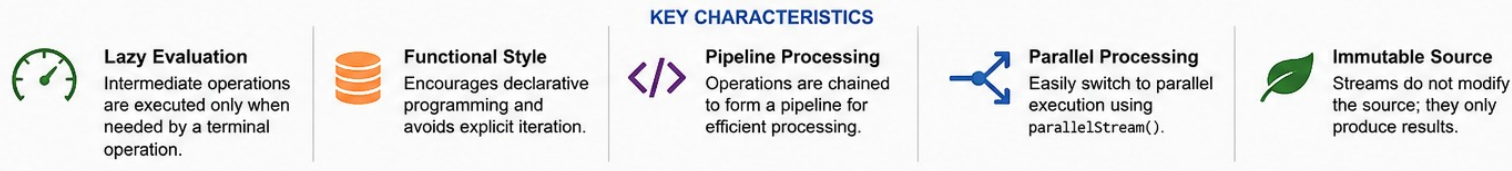
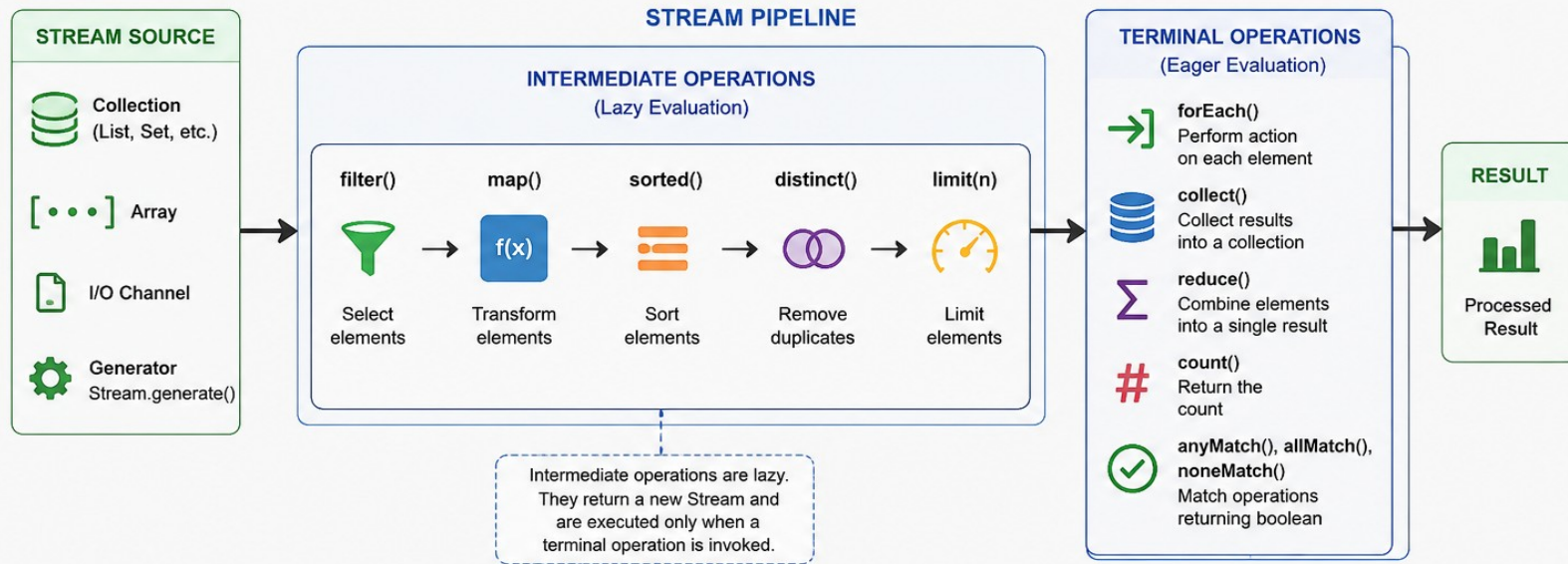
- Imperative: more code, more state, more room for errors.
- Functional: less code, easier to read, safer, and more expressive.

**KEY TAKEAWAY** Functional programming is not just a style — it leads to robust, scalable, future-ready software.



# JAVA STREAMS API

The Streams API (java.util.stream) provides a functional approach to process collections of data. It enables declarative, concise and efficient operations on data sequences.



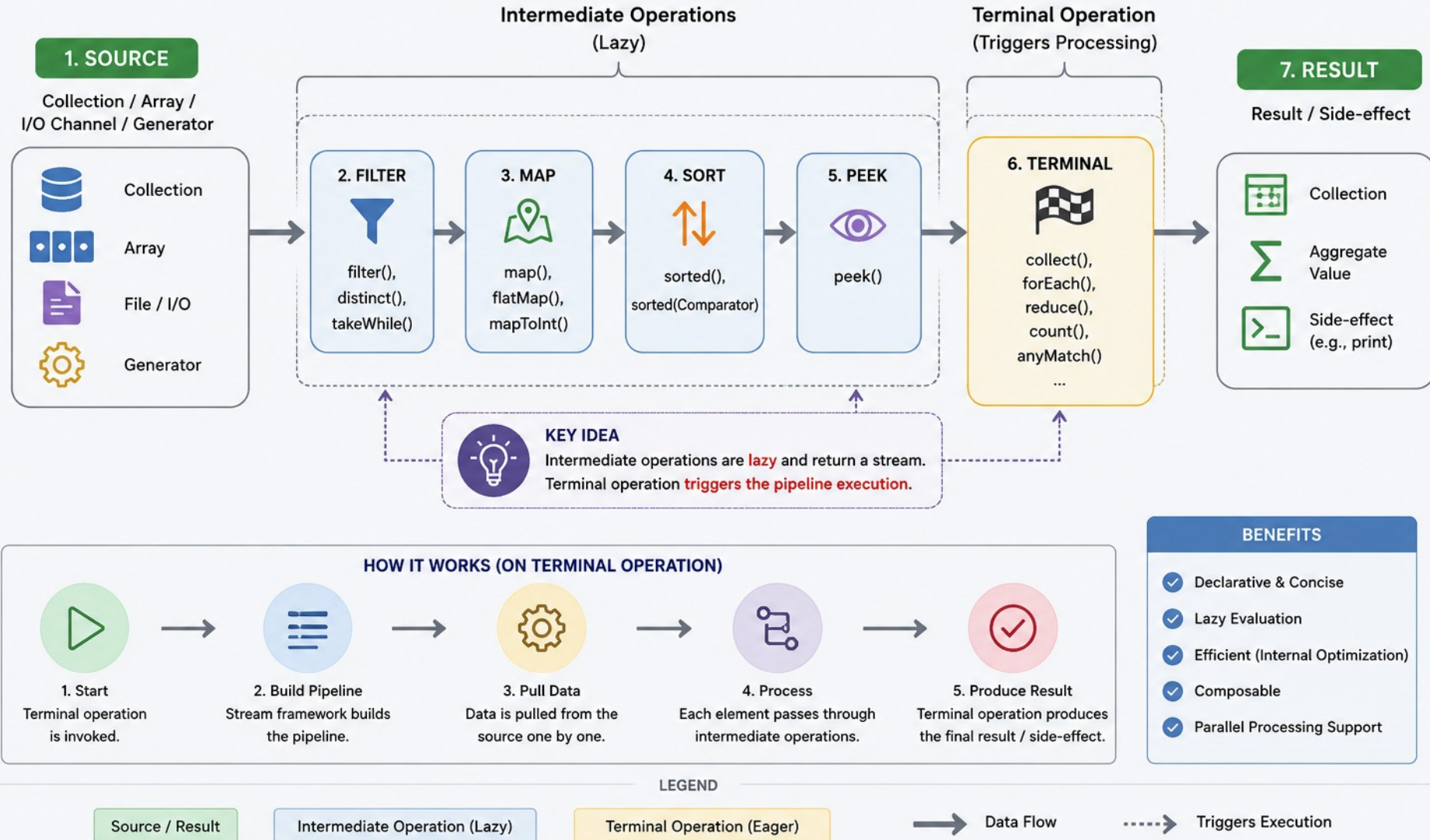
## EXAMPLE

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");  
List<String> result = names.stream().filter(n -> n.length() > 3).map(String::toUpperCase).sorted().collect(Collectors.toList());
```

**ALSO FROM THIS TOPIC** Single-use — a stream can be consumed only once • Not a data structure — no add/remove/update operations

# STREAM PROCESSING WORKFLOW (JAVA)

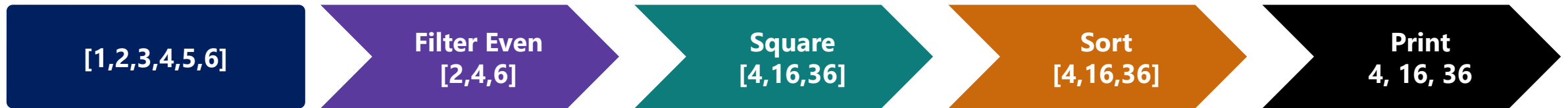
Streams API processes data in a declarative pipeline of operations.





# EXAMPLE SCENARIO

*From a list of numbers: get the evens, square them, sort, and print.*



```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
numbers.stream()
    .filter(n -> n % 2 == 0)           // intermediate
    .map(n -> n * n)                   // intermediate
    .sorted()                         // intermediate
    .forEach(System.out::println);    // terminal
```

**KEY TAKEAWAY** Every intermediate step in this diagram is lazy — only `forEach()` actually runs the pipeline.

# LAMBDA EXPRESSIONS IN JAVA

A lambda expression is a concise way to represent an anonymous function (a method without a name).

## SYNTAX

`(parameters) -> expression`  
or  
`(parameters) -> { statements; }`

## STRUCTURE OF A LAMBDA EXPRESSION

### PARAMETERS

Input to the lambda.  
Can be zero, one  
or more.



### ARROW TOKEN

Separates parameters  
and the body.



### BODY

Contains the expression  
or statements to be  
executed.

## EXAMPLES

### 1. NO PARAMETERS

```
() -> "Hello World"
```

No input, returns  
"Hello World"

### 2. SINGLE PARAMETER

```
x -> x * x
```

Takes one parameter x  
and returns its square

### 3. MULTIPLE PARAMETERS

```
(a, b) -> a + b
```

Takes two parameters  
and returns their sum

### 4. MULTIPLE STATEMENTS

```
(x) -> {  
    int y = x * 2;  
    return y;  
}
```

Body contains multiple  
statements

### 5. TYPE DECLARATION (OPTIONAL)

```
(int x) -> x + 10
```

Type declaration is  
optional. Compiler can  
infer the type.

## HOW LAMBDA EXPRESSION WORKS

### FUNCTIONAL INTERFACE

An interface with  
a single abstract  
method.  
e.g.,  
`java.util.function  
.Function <T, R>`

Target Type

### LAMBDA EXPRESSION

```
(x) -> x * x
```

Provides the  
implementation.

### RUNTIME

Lambda is converted  
to an instance of the  
functional interface.

### EXECUTION

The abstract method  
is invoked and the  
lambda body is  
executed.

### USES

- Collection operations
- Event handling
- Threads
- Stream API
- Callback functions



### BENEFITS



Concise and readable



Improves productivity



Enables functional  
programming



Reduces boilerplate code

# LAMBDA VS ANONYMOUS CLASS

*The same behavior, five different shapes — lambdas are always shorter.*

| SCENARIO                        | LAMBDA                                         | EQUIVALENT ANONYMOUS CLASS                               |
|---------------------------------|------------------------------------------------|----------------------------------------------------------|
| No params, return a value       | <code>() -&gt; 42</code>                       | <code>new Supplier&lt;Integer&gt;() {...}</code>         |
| One param, return a value       | <code>x -&gt; x * x</code>                     | <code>new Function&lt;Integer,Integer&gt;() {...}</code> |
| Multiple params, return a value | <code>(a, b) -&gt; a + b</code>                | <code>new BiFunction&lt;...&gt;() {...}</code>           |
| No params, no return (void)     | <code>() -&gt; System.out.println("Hi")</code> | <code>new Runnable() {...}</code>                        |
| One param, no return (void)     | <code>msg -&gt; System.out.println(msg)</code> | <code>new Consumer&lt;String&gt;() {...}</code>          |

**WHERE LAMBDA'S ARE USED** Collection processing, event handling, executors, custom APIs — anywhere Java expects a functional interface.

```
names.forEach(name -> System.out.println(name.toUpperCase())); // ALICE BOB CHARLIE
```

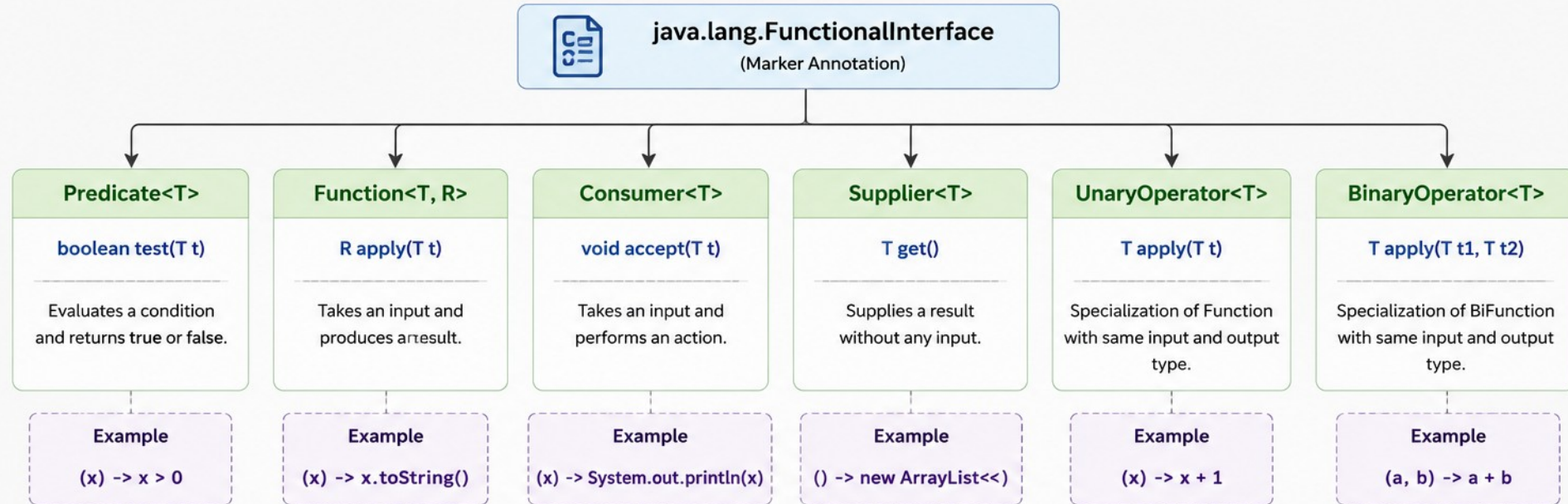
**KEY TAKEAWAY** Every lambda on the left compiles down to the same kind of object shown on the right — just with far less ceremony.



# FUNCTIONAL INTERFACES IN JAVA



A Functional Interface is an interface that contains **exactly one abstract method**. They can have any number of default, static, and Object methods.



## Key Characteristics

1

### Single Abstract Method

Exactly one abstract method (SAM) defines the contract.



### Lambda Compatible

Can be implemented using lambda expressions or method references.



### Used in Stream API

Heavily used in Java Streams for functional-style operations.



### @FunctionalInterface

Optional annotation that enables compile-time validation.

## Benefits

- ✓ Promotes functional programming
- ✓ Clean and concise code
- ✓ Improves readability
- ✓ Encourages code reusability



Examples of other functional interfaces: **BiFunction<T,U,R>** • **BiConsumer<T,U>** • **Runnable** • **Comparator<T>** • **Optional<T>**

# WHY THEY MATTER & COMMON INTERFACES

*The foundation that lets you pass behavior as data — enabling lambdas, method references, and the Streams API.*

| INTERFACE                            | TYPE     | ABSTRACT METHOD                     | DESCRIPTION                             |
|--------------------------------------|----------|-------------------------------------|-----------------------------------------|
| <code>Predicate&lt;T&gt;</code>      | Function | <code>boolean test(T t)</code>      | Evaluates a condition.                  |
| <code>Function&lt;T,R&gt;</code>     | Function | <code>R apply(T t)</code>           | Transforms input to output.             |
| <code>BiFunction&lt;T,U,R&gt;</code> | Function | <code>R apply(T t, U u)</code>      | Works on two inputs, produces a result. |
| <code>Consumer&lt;T&gt;</code>       | Consumer | <code>void accept(T t)</code>       | Consumes a value, returns no result.    |
| <code>Supplier&lt;T&gt;</code>       | Supplier | <code>T get()</code>                | Supplies a value, takes no input.       |
| <code>BiPredicate&lt;T,U&gt;</code>  | Function | <code>boolean test(T t, U u)</code> | Evaluates a condition on two inputs.    |

**KEY TAKEAWAY** Predicate, Function, Consumer, and Supplier cover almost every functional-interface need in everyday code.

# USING A FUNCTIONAL INTERFACE WITH A LAMBDA

*The lambda supplies the implementation of the single abstract method.*

```
MyFunctionalInterface obj =  
    (message) -> System.out.println("Hello " + message);  
  
obj.perform("Java");    // Output: Hello Java
```

Functional interfaces are the foundation of Java's functional programming. They allow behavior to be passed as an argument, enabling powerful and expressive code with lambdas, method references, and the Streams API.

**Declaring your own:** `@FunctionalInterface public interface Name { returnType method(params); }`

Default and static methods are optional extras — only the single abstract method counts toward the rule.

**KEY TAKEAWAY** The lambda expression provides the implementation for the abstract method defined in the functional interface.



# TRY IT YOURSELF — EXERCISE 1: LAMBDA EXPRESSIONS

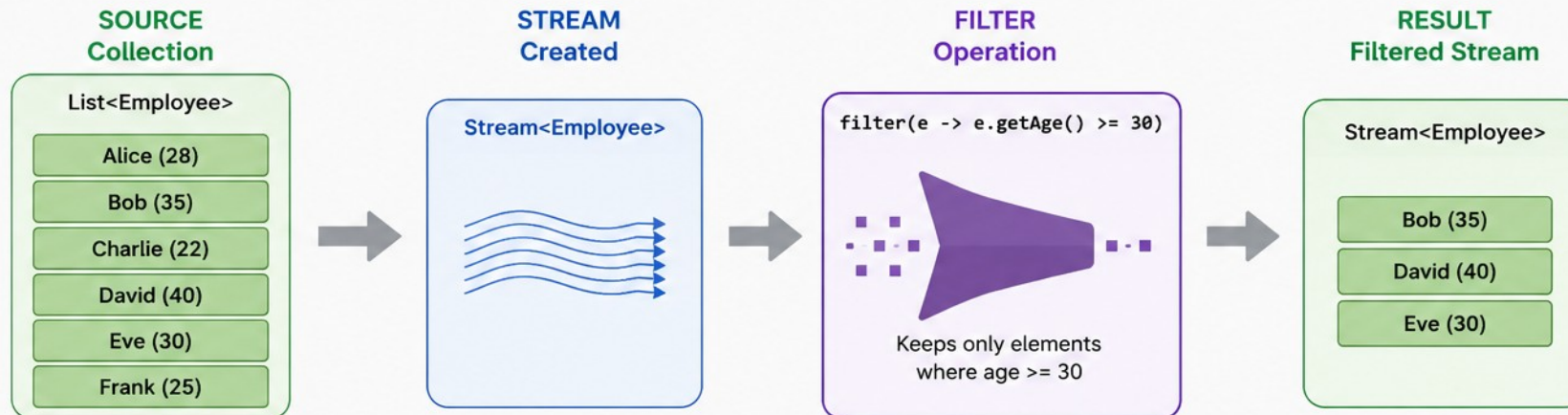
*Reinforces: lambdas and functional interfaces, as just covered.*

| STEP        | CODE                                                                   | RESULT                                                        |
|-------------|------------------------------------------------------------------------|---------------------------------------------------------------|
| Goal        | Implement a one-method interface as an anonymous class and as a lambda | Both produce the same result                                  |
| Code        | <code>SalaryCheck lambda = e -&gt; e.salary() &gt; 60_000;</code>      | A lambda implementing <code>SalaryCheck.test(Employee)</code> |
| Key concept | A functional interface has exactly one abstract method                 | That single method is what a lambda implements                |
| Reference   | <code>exercises/exercise-01-lambda-functional-interface.md</code>      | Full starter code and steps                                   |

**TRY IT NOW** Complete Exercise 1 before continuing — see `exercises/exercise-01-lambda-functional-interface.md`.

# FILTERING DATA WITH STREAMS (JAVA)

The `filter()` intermediate operation selects elements that match a given **predicate (condition)** and returns a new Stream containing only those elements.



## EXAMPLE CODE

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", 28),
    new Employee("Bob", 35),
    new Employee("Charlie", 22),
    new Employee("David", 40),
    new Employee("Eve", 30),
    new Employee("Frank", 25)
);
List<Employee> filtered = employees.stream()
    .filter(e -> e.getAge() >= 30) // Filtering condition
    .collect(Collectors.toList()); // Collect results
```

## KEY POINTS

-  `filter()` is an intermediate operation.
-  Takes a `Predicate<T>` and returns a `Stream<T>`.
-  Does not modify the original data.
-  Supports method references and lambda expressions.

## PREDICATE EXAMPLES

### Using Lambda Expression

```
.filter(e -> e.getAge() >= 30)
```

### Using Method Reference (if applicable)

```
.filter(Employee::isActive)
// where isActive() returns boolean
```

### Compound Condition

```
.filter(e -> e.getAge() >= 30 && e.getSalary() > 50000)
```

# REAL-WORLD EXAMPLE: FILTER EMPLOYEES BY SALARY

*Keep only employees earning more than 60,000.*

| NAME    | DEPARTMENT | SALARY |
|---------|------------|--------|
| Alice   | IT         | 85000  |
| Bob     | HR         | 45000  |
| Charlie | IT         | 95000  |
| David   | Finance    | 55000  |
| Eve     | IT         | 90000  |

```
employees.stream()
    .filter(e ->
        e.getSalary() > 60000)
    .collect(
        Collectors.toList());

// Alice, Charlie, Eve
```

**KEY TAKEAWAY** Use `filter()` to narrow your data down to only the elements that match a condition — the foundation of stream processing.

# TRY IT YOURSELF — EXERCISE 2: FILTER BY SALARY

*Reinforces: the filter operation you just saw.*

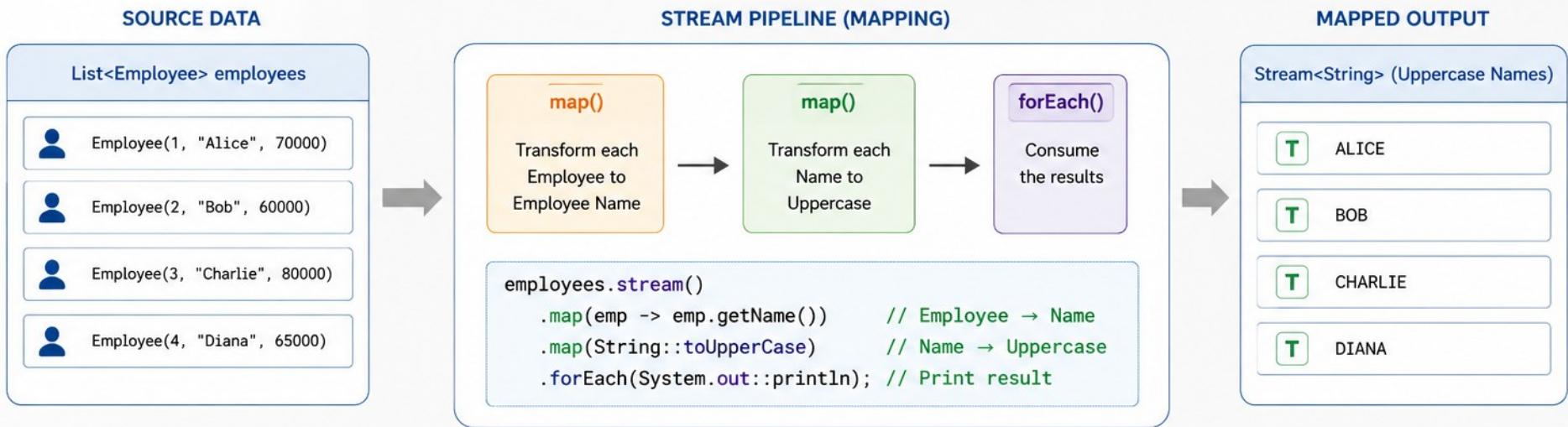
| STEP        | CODE                                                                   | RESULT                                        |
|-------------|------------------------------------------------------------------------|-----------------------------------------------|
| Goal        | List employees with salary greater than 60,000                         | Alice, Bob, Charlie, Diana (not Evan)         |
| Code        | <code>.filter(e -&gt; e.salary() &gt; 60_000)</code>                   | A shorter stream — Evan is excluded           |
| Key concept | <code>filter()</code> keeps only elements matching a boolean predicate | Ties to the filtering operations you just saw |
| Reference   | <code>exercises/exercise-02-filter-salary.md</code>                    | Full starter code and steps                   |

**TRY IT NOW** Complete Exercise 2 before continuing — see `exercises/exercise-02-filter-salary.md`.

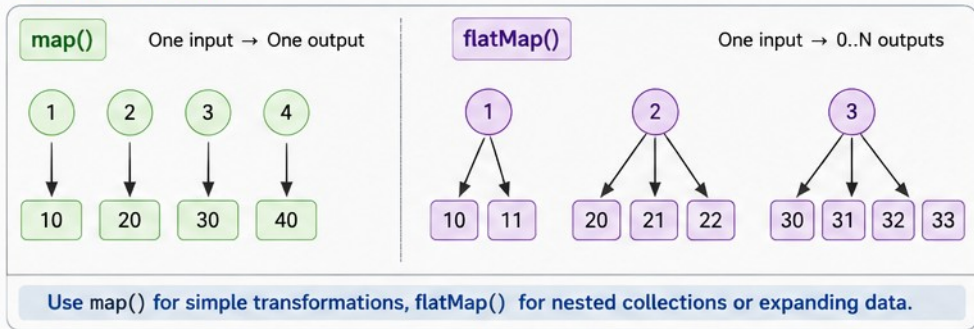


# MAPPING DATA WITH STREAMS

Mapping transforms each element of a stream to another object using the `map()` or `flatMap()` operations.



## map() vs flatMap()



## COMMON MAPPING EXAMPLES

| Goal                       | Example                                                                            |
|----------------------------|------------------------------------------------------------------------------------|
| Extract specific field     | <code>employees.stream().map(Employee::getName)</code>                             |
| Convert to another type    | <code>employees.stream().map(Employee::getId)</code>                               |
| Compute derived value      | <code>employees.stream().map(emp -&gt; emp.getSalary() * 1.1)</code>               |
| Flatten nested collections | <code>departments.stream().flatMap(dept -&gt; dept.getEmployees().stream())</code> |

BEST PRACTICES

✓ Use map() for one-to-one transformation.

✓ Use flatMap() to flatten nested streams or collections.

✓ Keep mapping functions stateless and side-effect free.

✓ Chain multiple map() operations for complex transformations.

# EXAMPLE: GET LENGTH OF EACH STRING

*A second mapping example, plus where map() shows up in practice.*

```
List<Integer> lengths = names.stream()  
    .map(name -> name.length())    // length of each string  
    .collect(Collectors.toList()); // [5, 3, 7]
```

- Common use cases: data transformation, format conversion, extracting properties,
- calculations, and preparing data for other operations in the pipeline.

**KEY TAKEAWAY** Use map() to transform each element into a new form — a powerful tool for shaping data for the next pipeline step.

# TRY IT YOURSELF — EXERCISE 3: LIST ALL NAMES

*Reinforces: the map operation you just saw.*

| STEP        | CODE                                                           | RESULT                                      |
|-------------|----------------------------------------------------------------|---------------------------------------------|
| Goal        | Print all employee names using map +<br>forEach or collect     | All five names print                        |
| Code        | <code>.map(Employee::name).forEach(System.out::println)</code> | Alice, Bob, Charlie, Diana, Evan            |
| Key concept | map() transforms each element — Employee<br>to String here     | Ties to the mapping operations you just saw |
| Reference   | <code>exercises/exercise-03-list-names.md</code>               | Full starter code and steps                 |

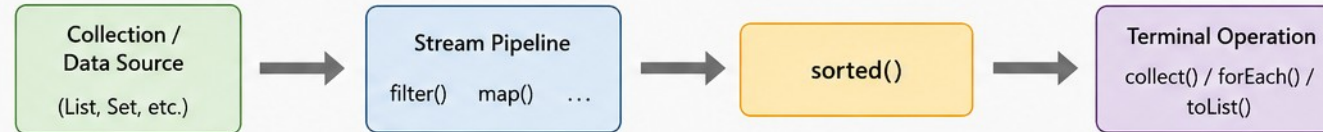
**TRY IT NOW** Complete Exercise 3 before continuing — see `exercises/exercise-03-list-names.md`.

# SORTING WITH STREAMS IN JAVA

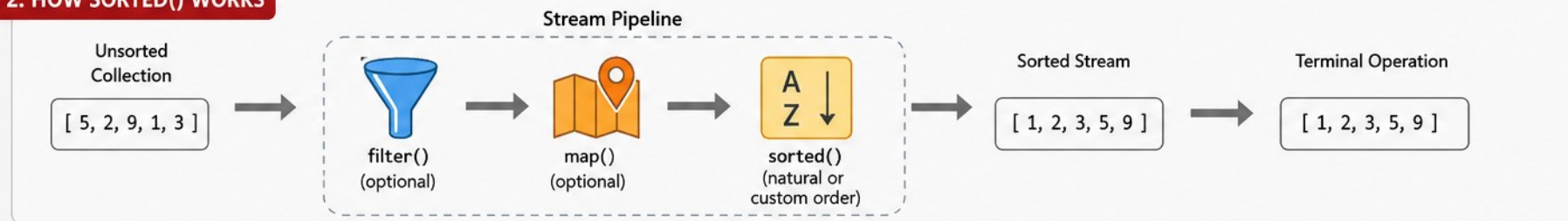
The Stream API provides the `sorted()` operation to sort elements in natural order or using a custom Comparator.

## 1. OVERVIEW

`sorted()` is an intermediate operation that returns a stream with elements sorted in natural order or by a provided Comparator.



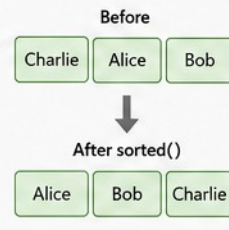
## 2. HOW SORTED() WORKS



## 3. SORTING EXAMPLES

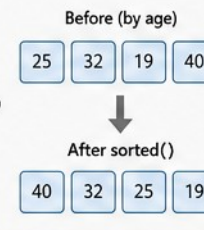
### A. Natural Sorting (Comparable)

```
List<String> names = List.of("Charlie", "Alice", "Bob");  
  
List<String> sorted = names.stream()  
    .sorted()  
    .toList();  
  
// Output: [Alice, Bob, Charlie]
```



### B. Custom Sorting (Comparator)

```
List<Person> people = ...;  
  
List<Person> sorted = people.stream()  
    .sorted(Comparator.comparing(Person::getAge)  
        .reversed()) // Descending by age  
    .toList();  
  
// Output: Sorted by age (highest to lowest)
```



## COMPARATOR EXAMPLES

- Natural Order  
`sorted()`
- Reverse Order  
`sorted(Comparator.reverseOrder())`
- By Property  
`sorted(Comparator.comparing(Person::getAge))`
- Multiple Fields  
`sorted(Comparator.comparing(Person::getAge).thenComparing(Person::getName))`

## 4. KEY POINTS



`sorted()` is stable (maintains order of equal elements).



Natural sorting requires elements to implement Comparable.



Use Comparator for custom sorting logic.



Can be combined with other operations in the pipeline.



Returns a new sorted stream; original data remains unchanged.



# EXAMPLE: SORT OBJECTS BY SALARY (DESCENDING)

*Comparator.comparing(...).reversed() sorts custom objects in descending order.*

```
List<Employee> sortedBySalary = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary)
        .reversed())           // descending by salary
    .collect(Collectors.toList());
```

| NAME    | SALARY (DESC) |
|---------|---------------|
| Bob     | 95000         |
| Alice   | 75000         |
| Charlie | 55000         |

## MORE SORTING PATTERNS

`sorted()` — natural order

`sorted(Comparator.reverseOrder())` — reverse order

`sorted(String.CASE_INSENSITIVE_ORDER)` — case-insensitive

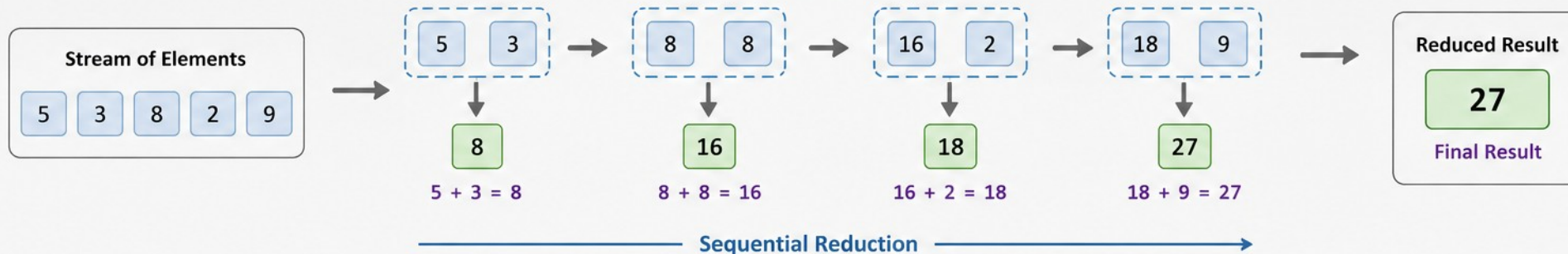
`sorted(Comparator.comparing(p -> p.name()))` — by property

**KEY TAKEAWAY** Use `sorted()` with natural ordering or a custom `Comparator` for flexible, expressive sorting.

# REDUCTION OPERATIONS IN JAVA

Reduction operations combine the elements of a stream into a single result using an identity value and an associative accumulation function.

## HOW REDUCTION WORKS



### TYPES OF REDUCTION (Stream API)

- 1 reduce(identity, accumulator)**  
Combines elements with an identity value. Returns identity if stream is empty.  
`Optional<T> reduce(T identity, BinaryOperator<T> accumulator)`
- 2 reduce(accumulator)**  
Combines elements without an identity. Returns `Optional.empty()` if stream is empty.  
`Optional<T> reduce(BinaryOperator<T> accumulator)`
- 3 reduce(identity, accumulator, combiner)**  
Used in parallel streams. Combines partial results using combiner.  
`T reduce(T identity, BinaryOperator<T> accumulator, BinaryOperator<T> combiner)`

### COMMON EXAMPLES

|  |                                                                           | Result  |
|--|---------------------------------------------------------------------------|---------|
|  | <b>Sum</b><br><code>reduce(0, Integer::sum)</code>                        | 27      |
|  | <b>Product</b><br><code>reduce(1, (a, b) -&gt; a * b)</code>              | 2160    |
|  | <b>Maximum</b><br><code>reduce(Integer::max)</code>                       | 9       |
|  | <b>Minimum</b><br><code>reduce(Integer::min)</code>                       | 2       |
|  | <b>Concatenate Strings</b><br><code>reduce("", (a, b) -&gt; a + b)</code> | "53829" |

### CODE EXAMPLE

```
List<Integer> numbers = Arrays.asList(5, 3, 8, 2, 9);

// 1. With identity
int sum = numbers.stream()
    .reduce(0, Integer::sum);
System.out.println("Sum = " + sum); // 27

// 2. Without identity
Optional<Integer> max = numbers.stream()
    .reduce(Integer::max);
max.ifPresent(m -> System.out.println("Max = " + m)); // 9

// 3. Parallel stream with combiner
int sumParallel = numbers.parallelStream()
    .reduce(0, Integer::sum, Integer::sum);
System.out.println("Parallel Sum = " + sumParallel); // 27
```



### KEY POINTS

- Reduction produces a single summary result from the stream.
- The accumulator function must be associative.
- Use identity when a default starting value exists.
- For parallel streams, provide a proper combiner function.



Works with  
Sequential &  
Parallel Streams

# REDUCTION METHODS OVERVIEW

*Eight methods, each with a distinct return type.*

| METHOD                                         | RETURN TYPE     | DESCRIPTION                           | EXAMPLE                                     |
|------------------------------------------------|-----------------|---------------------------------------|---------------------------------------------|
| <code>reduce(identity, accumulator)</code>     | T               | Combines using identity + accumulator | <code>reduce(0, Integer::sum)</code>        |
| <code>reduce(accumulator)</code>               | Optional<T>     | Combines without an identity value    | <code>reduce(Integer::sum)</code>           |
| <code>sum()</code>                             | int/long/double | Sum of numeric stream elements        | <code>mapToInt(i-&gt;i).sum()</code>        |
| <code>min(comparator) / max(comparator)</code> | Optional<T>     | Smallest / largest element            | <code>min(Comparator.naturalOrder())</code> |
| <code>count()</code>                           | long            | Number of elements                    | <code>count()</code>                        |
| <code>average()</code>                         | OptionalDouble  | Average of numeric elements           | <code>mapToInt(i-&gt;i).average()</code>    |
| <code>collect(collector)</code>                | R               | Reduces using a Collector             | <code>collect(Collectors.toList())</code>   |

**KEY TAKEAWAY** Know your return types: Optional<T>, OptionalDouble, long, int — each reduction returns a different shape.

# REDUCTION — CODE EXAMPLES

*reduce, min, max, count, and average, all on the same data.*

```
List<Integer> numbers = Arrays.asList(2, 4, 6, 8, 10);

int sum = numbers.stream().reduce(0, Integer::sum);           // 30
Optional<Integer> sumOpt = numbers.stream().reduce(Integer::sum); // Optional[30]
Optional<Integer> min = numbers.stream().min(Integer::compare); // Optional[2]
Optional<Integer> max = numbers.stream().max(Integer::compare); // Optional[10]
long count = numbers.stream().count();                       // 5
OptionalDouble avg = numbers.stream()
    .mapToInt(Integer::intValue).average(); // OptionalDouble[6.0]

// Custom reduction (no built-in method covers this):
String csv = names.stream().reduce("", (a, b) -> a.isEmpty() ? b : a + ", " + b); // Alice, Bob, Charlie
```

**KEY TAKEAWAY** `reduce(identity, accumulator)` always returns `T`; `reduce(accumulator)` returns `Optional<T>` to handle empty streams.



# REDUCTION — BEST PRACTICES

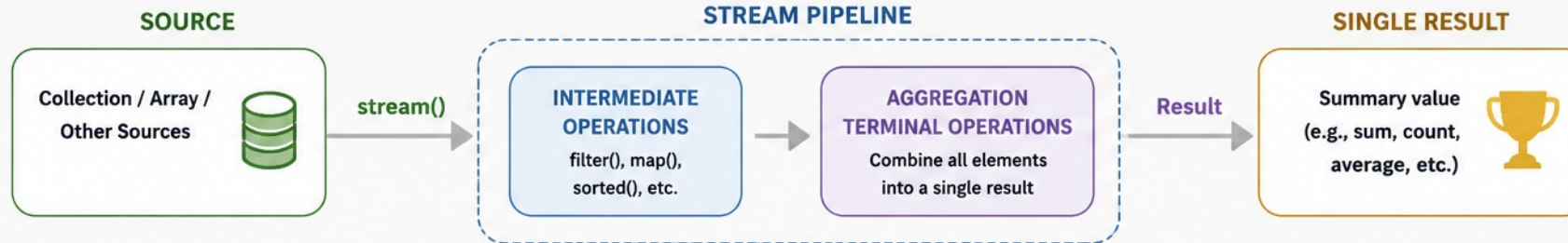
*Five guidelines for choosing and writing reductions correctly.*

- Use built-in reduction methods (sum, min, max, count, average) whenever possible.
- Use `reduce(identity, accumulator)` when a natural identity exists (0 for sum, 1 for product, "" for concatenation).
- Use `reduce(accumulator)` when no identity value exists or when working with objects.
- Ensure the accumulator is associative and stateless for correct parallel execution.
- Prefer `Optional<T>` results to safely handle empty streams.

**KEY TAKEAWAY** Think in terms of identity, accumulator, and associativity — reduction operations are essential for aggregation and summarization.

# AGGREGATION OPERATIONS IN JAVA STREAMS

Combine multiple elements into a single summary result



## COMMON AGGREGATION OPERATIONS



### SUM

Returns the sum of numeric elements.

```
int sum = numbers.stream()
    .mapToInt(Integer::intValue)
    .sum();
```



### COUNT

Returns the count of elements.

```
long count = items.stream()
    .count();
```



### AVERAGE

Returns the average of numeric elements.

```
double avg = numbers.stream()
    .mapToDouble(Integer::doubleValue)
    .average()
    .orElse(0.0);
```



### MIN

Returns the minimum element.

```
Optional<Integer> min = numbers.stream()
    .min(Integer::compare);
```



### MAX

Returns the maximum element.

```
Optional<Integer> max = numbers.stream()
    .max(Integer::compare);
```



### REDUCE

Combines elements using an accumulator function.

```
int total = numbers.stream()
    .reduce(0, (a, b) -> a + b);
```



### COLLECT

Aggregates elements into a collection or summary object.

```
List<String> list = names.stream()
    .collect(Collectors.toList());
```



### GROUPING & SUMMARIZING (Collectors)

Performs advanced aggregations like grouping, partitioning, and summarizing.

```
Map<String, Long> byCity = people.stream()
    .collect(Collectors.groupingBy(
        Person::getCity, Collectors.counting()));
```



### Key Points

- Aggregation operations produce a single result.
- They are terminal operations (pipeline ends here).

- Used for calculations, summaries, and data analysis.
- Provide better readability and concise code.



# HOW AGGREGATION WORKS

*Given the stream 2, 4, 6, 8, 10 — every metric computed in one pass.*

| METRIC  | VALUE |
|---------|-------|
| Count   | 5     |
| Sum     | 30    |
| Min     | 2     |
| Average | 6.0   |
| Max     | 10    |

```
IntSummaryStatistics stats =  
    numbers.stream()  
        .mapToInt(i -> i)  
        .summaryStatistics();  
  
stats.getCount();    // 5  
stats.getSum();      // 30  
stats.getMin();      // 2  
stats.getAverage();  // 6.0  
stats.getMax();      // 10  
  
// Custom: a Collector-based aggregation  
employees.stream().collect(  
    summingDouble(Employee::getSalary)); // 225000.0
```

**KEY TAKEAWAY** Aggregation condenses many values into meaningful insights that help in decision making.

# AGGREGATION — BEST PRACTICES

*Five guidelines for accurate, efficient aggregation.*

- Choose the right aggregation method for the data type.
- Use `mapToInt()`, `mapToLong()`, or `mapToDouble()` before `sum()`/`average()` for numeric streams.
- Handle Optional results from `min()`, `max()`, and `average()` safely.
- Use `summaryStatistics()` when multiple metrics are needed together.
- Aggregation operations are terminal — they produce a single result.

**KEY TAKEAWAY** Aggregation helps in analytics, reporting, and decision making — pick the right operation and return type.



# TRY IT YOURSELF — EXERCISE 4: HIGHEST AND LOWEST SALARY

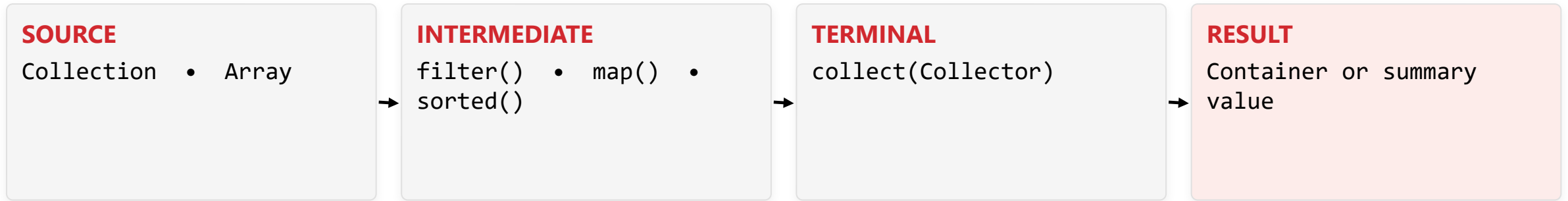
*Reinforces: the aggregation operations you just saw.*

| STEP        | CODE                                                       | RESULT                                          |
|-------------|------------------------------------------------------------|-------------------------------------------------|
| Goal        | Find max and min salary with<br>Comparator.comparingDouble | Max Diana 90000; min Evan 55000                 |
| Code        | .max(comparingDouble(Employee::salary)).orElseThrow()      | max()/min() return an Optional<Employee>        |
| Key concept | orElseThrow() unwraps the Optional or fails fast if empty  | Ties to the aggregation operations you just saw |
| Reference   | exercises/exercise-04-minmax.md                            | Full starter code and steps                     |

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-minmax.md.

# COLLECTORS FRAMEWORK

*A rich set of collector implementations to accumulate stream elements into result containers or summary values.*



- Collectors are immutable, stateless, and thread-safe — all defined in `java.util.stream.Collectors`.
- Common use cases: grouping, partitioning, summarizing, joining, and collecting into collections.

**KEY TAKEAWAY** `collect(Collector)` is the terminal operation that triggers the pipeline and returns the result.

# HOW COLLECTORS WORK

*Collectors.toSet() on ["Alice", "Bob", "Charlie", "Bob"] — duplicates are removed.*



```
List<Integer> list = numbers.stream().collect(Collectors.toList());    // [1,2,3,4,5]
Set<Integer> set = numbers.stream().collect(Collectors.toSet());      // {1,2,3,4,5}
String joined = numbers.stream().map(Object::toString)
    .collect(Collectors.joining(", "));    // "1, 2, 3, 4, 5"
```

**KEY TAKEAWAY** A collector defines how to accumulate elements, the result type, and how partial results combine in parallel.

# COMMON COLLECTORS — PART 1

*Collection and reduction collectors.*

| CATEGORY   | COLLECTOR                           | DESCRIPTION                         | EXAMPLE                         |
|------------|-------------------------------------|-------------------------------------|---------------------------------|
| Collection | toList() / toSet()                  | Collects into a List / Set          | collect(toList())               |
| Collection | toCollection(Supplier)              | Collects into a specific collection | toCollection(ArrayList::new)    |
| Reduction  | counting()                          | Counts the number of elements       | collect(counting())             |
| Reduction  | summingInt() /<br>averagingDouble() | Sums / averages values              | summingInt(Employee::getSalary) |
| Reduction  | minBy() / maxBy()                   | Min/max element using a Comparator  | minBy(naturalOrder())           |

**KEY TAKEAWAY** count=5, sum=15, min=1, average=3.0, max=5 — that's what `Collectors.summarizingInt()` gives you in one call.



# COMMON COLLECTORS — PART 2 & GROUPING EXAMPLE

*Grouping and joining collectors, plus groupingBy() in action.*

| CATEGORY | COLLECTOR                      | DESCRIPTION                              | EXAMPLE                              |
|----------|--------------------------------|------------------------------------------|--------------------------------------|
| Grouping | groupingBy(...)                | Groups elements by a classifier function | groupingBy(Employee::getDepartment)  |
| Grouping | partitioningBy(...)            | Partitions into two groups (true/false)  | partitioningBy(e -> e.getAge() > 30) |
| Joining  | joining(delim, prefix, suffix) | Joins with delimiter, prefix, and suffix | joining(", ", "[", "]")              |

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Result: HR -> [Alice, Charlie]  IT -> [Bob, David]
```

**KEY TAKEAWAY** groupingBy() is the collector you'll reach for most in real business reporting.

# TRY IT YOURSELF — EXERCISE 5: MAP 10% RAISE

*Reinforces: map() and collecting a stream into a List, as just covered.*

| STEP        | CODE                                                           | RESULT                                        |
|-------------|----------------------------------------------------------------|-----------------------------------------------|
| Goal        | Map salaries ×1.10, collect into a List<Double>                | Updated salary list prints                    |
| Code        | .map(e -> e.salary() * 1.10).toList()                          | A new List<Double> of raised salaries         |
| Key concept | toList() is a terminal op that collects the stream into a List | Ties to the Collectors framework you just saw |
| Reference   | exercises/exercise-05-map-raise.md                             | Full starter code and steps                   |

**TRY IT NOW** Complete Exercise 5 before continuing — see exercises/exercise-05-map-raise.md.

# TRY IT YOURSELF — EXERCISE 6: COUNT BY DEPARTMENT

*Reinforces: groupingBy and counting, as just covered.*

| STEP        | CODE                                                         | RESULT                                        |
|-------------|--------------------------------------------------------------|-----------------------------------------------|
| Goal        | Count employees per department with<br>groupingBy + counting | Finance=1, HR=2, IT=2 (TreeMap: alphabetical) |
| Code        | groupingBy(Employee::department,<br>counting())              | A Map<String, Long> of counts                 |
| Key concept | groupingBy buckets elements by a classifier<br>function      | Ties to the Collectors framework you just saw |
| Reference   | exercises/exercise-06-group-count.md                         | Full starter code and steps                   |

**TRY IT NOW** Complete Exercise 6 before continuing — see [exercises/exercise-06-group-count.md](#).

# STREAM PIPELINES

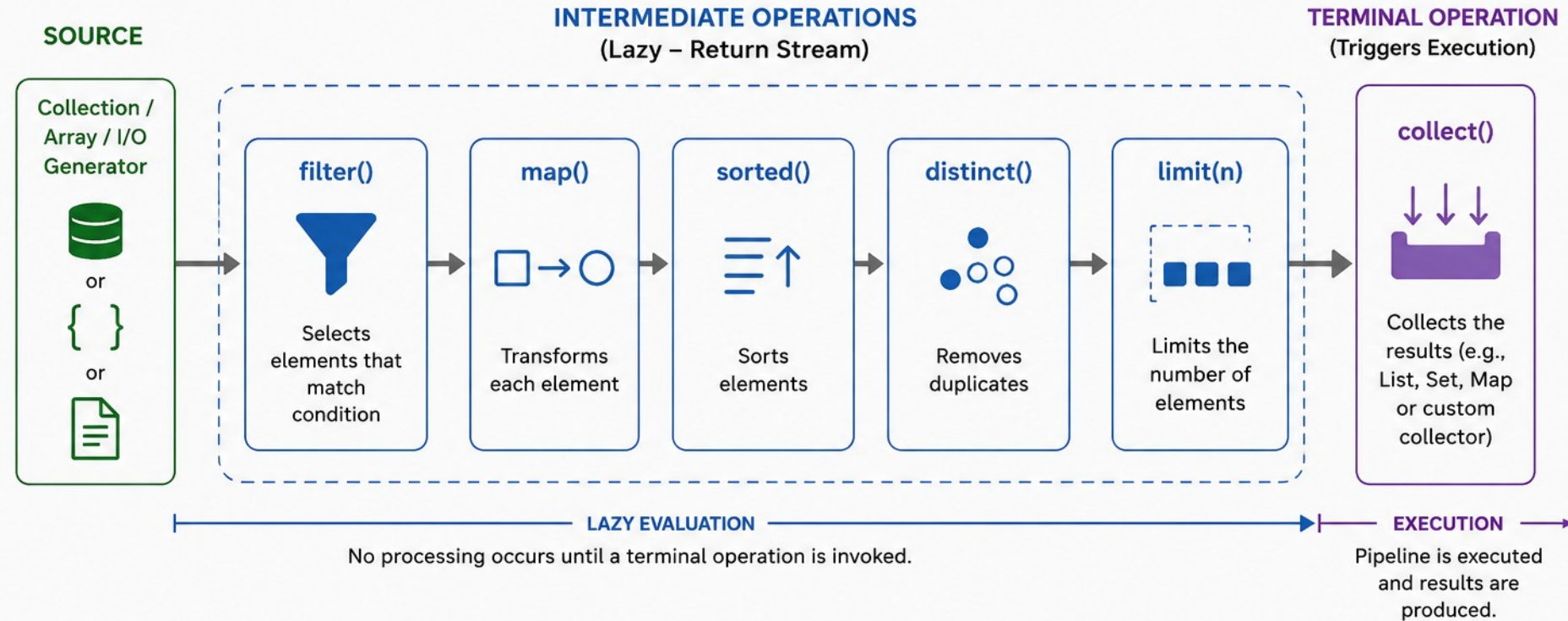
*A sequence of operations on a stream: a source, zero or more intermediate operations, and a terminal operation.*

| STAGE                  | PURPOSE                                               |
|------------------------|-------------------------------------------------------|
| Source                 | Get a stream from the data source.                    |
| Intermediate Operation | Transform or filter the stream (lazy).                |
| Intermediate Operation | Apply additional transformations or filtering (lazy). |
| Terminal Operation     | Produce a result or side effect (eager).              |
| Result                 | Collection, value, or side effect.                    |

**KEY TAKEAWAY** Pipelines are designed for readability, composability, and parallelism — and never modify the source data.

# STREAM PIPELINES IN JAVA

A Stream Pipeline consists of a data source, zero or more intermediate operations, and a terminal operation. Intermediate operations are lazy and return a stream. The terminal operation triggers the pipeline execution.



## Lazy Evaluation

Intermediate operations do not process data until needed.



## Pipelining

Operations are linked together to form a pipeline.



## Single Pass

Data flows through the pipeline once, improving performance.



## Optimization

Streams enable internal optimizations like short-circuiting.

## Example Pipeline

```
List<String> names = ...
names.stream()
    .filter(s -> s.startsWith("A"))
    .map(String::toUpperCase)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```



# INTERMEDIATE VS TERMINAL OPERATIONS

*Two very different roles in the same pipeline.*



## Intermediate Operations

- Return another stream.
- Can be chained.
- Lazy — not executed until a terminal operation is called.
- Examples: filter(), map(), sorted(), distinct(), limit()



## Terminal Operations

- Produce a result or side effect.
- End the pipeline.
- Trigger processing of all operations in the pipeline.
- Examples: forEach(), collect(), count(), reduce(), findFirst()

### TERMINAL OPERATIONS BY CATEGORY

**Consumption:** forEach() | **Aggregation:** count(), sum(), min(), max(), reduce() | **Search:** findFirst(), findAny() | **Match:** anyMatch(), allMatch(), noneMatch() | **Materialization:** collect(), toList()

**KEY TAKEAWAY** Intermediate operations are lazy; terminal operations are eager — this is the single most important distinction in this module.

# TRY IT YOURSELF — EXERCISE 7: HR DEPARTMENT NAMES

*Reinforces: chaining filter and map into one pipeline.*

| STEP        | CODE                                                        | RESULT                                    |
|-------------|-------------------------------------------------------------|-------------------------------------------|
| Goal        | Get the names of employees in the HR department             | Alice, Charlie                            |
| Code        | <code>filter(department == HR) → map(name)</code>           | filter narrows first, then map transforms |
| Key concept | Multiple intermediate ops can chain before the terminal one | Ties to the pipeline example you just saw |
| Reference   | <code>exercises/exercise-07-hr-names.md</code>              | Full starter code and steps               |

**TRY IT NOW** Complete Exercise 7 before continuing — see `exercises/exercise-07-hr-names.md`.

# EXERCISE 7 — SOLUTION WALKTHROUGH

*The full worked solution to Exercise 7 — filter, map, sort, and collect in one pipeline.*

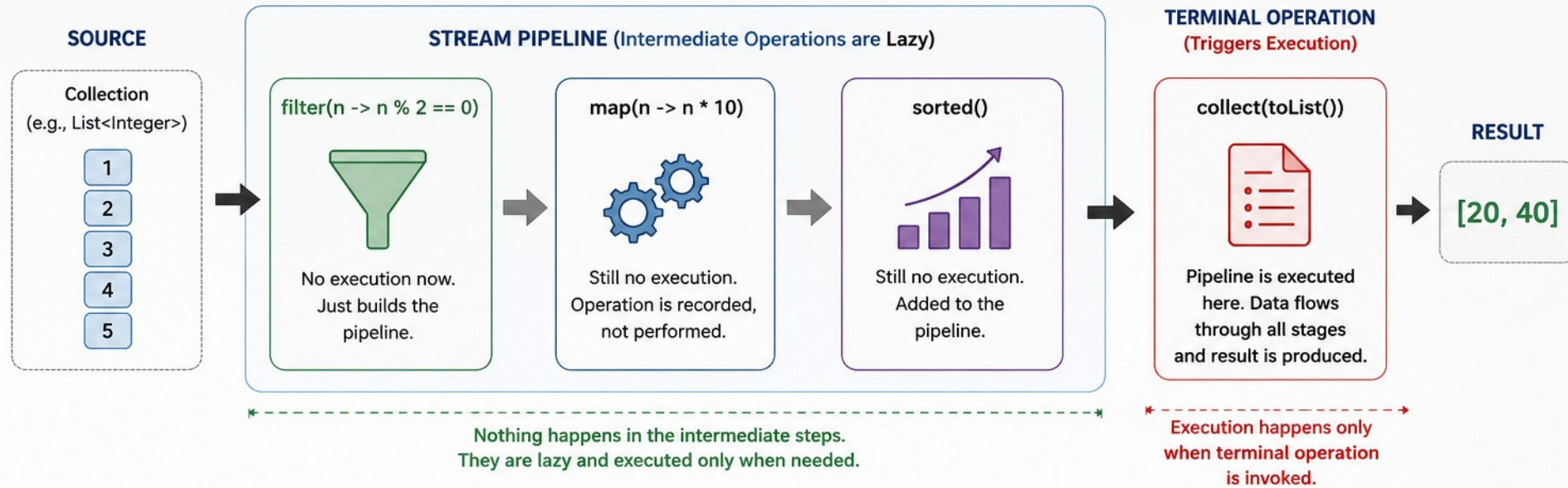
| STAGE                  | ACTION                                     |
|------------------------|--------------------------------------------|
| Source                 | employees (List<Employee>)                 |
| <code>filter()</code>  | Keep only employees from the HR department |
| <code>map()</code>     | Extract employee names                     |
| <code>sorted()</code>  | Sort names in natural order                |
| <code>collect()</code> | Collect into a List<String> (Result)       |

```
List<String> hrNames = employees.stream()
    .filter(e -> e.getDept().equals("HR")).map(Employee::getName)
    .sorted().collect(Collectors.toList()); // [Alice, Charlie]
```

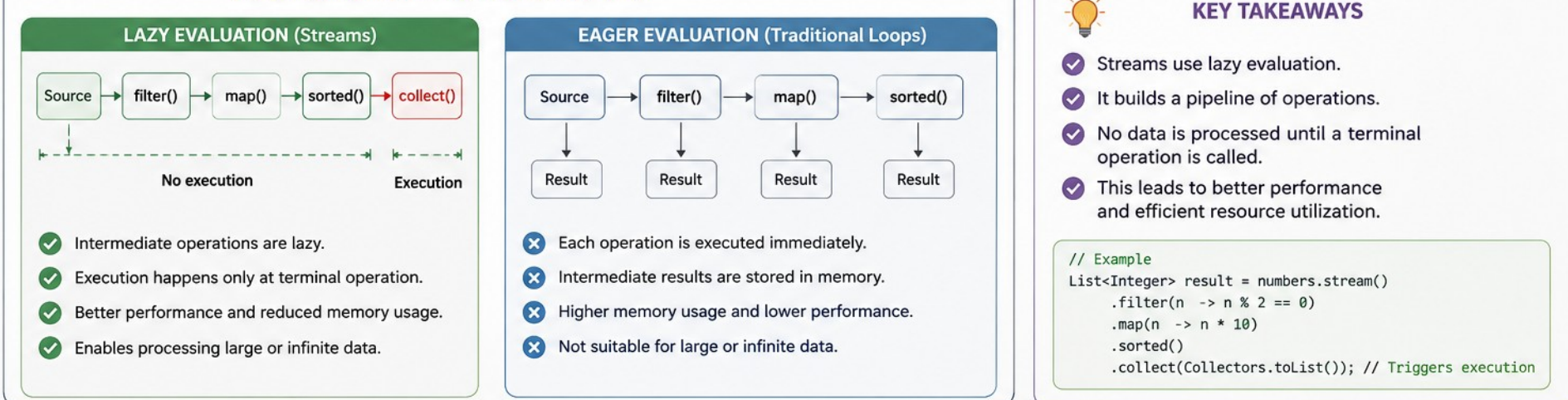
**KEY TAKEAWAY** The terminal operation `collect()` is what triggers the entire pipeline to actually run.

# LAZY EVALUATION IN JAVA

Intermediate operations are not executed until a terminal operation is invoked.



## LAZY EVALUATION vs EAGER EVALUATION



# PIPELINE ANALOGY

*Imagine a water pipeline: pipes are connected, but water doesn't flow until the tap opens.*



- Pipes are connected when the pipeline is built — but nothing flows yet.
- Water (data) only flows once `collect()` opens the tap.

**KEY TAKEAWAY** Pipes are connected when the pipeline is built; water only flows when `collect()` opens the tap.



# CODE EXAMPLE & OUTPUT

*Print statements inside filter()/map() reveal exactly when each element is processed.*

```
numbers.stream()
    .filter(n -> {
        System.out.println("filter: " + n);
        return n % 2 == 0;
    })
    .map(n -> {
        System.out.println("map: " + n);
        return n * 10;
    })
    .sorted();    // no terminal op yet

result.collect(Collectors.toList());
```

```
--- Building pipeline ---
--- Pipeline built, no output above ---
--- Now invoking terminal operation ---
filter: 1
filter: 2
map: 2
filter: 3
filter: 4
map: 4
filter: 5
Result: [20, 40]
```

**KEY TAKEAWAY** Not a single print statement runs until collect() is called — proof that intermediate operations are lazy.

# SHORT-CIRCUITING BENEFITS

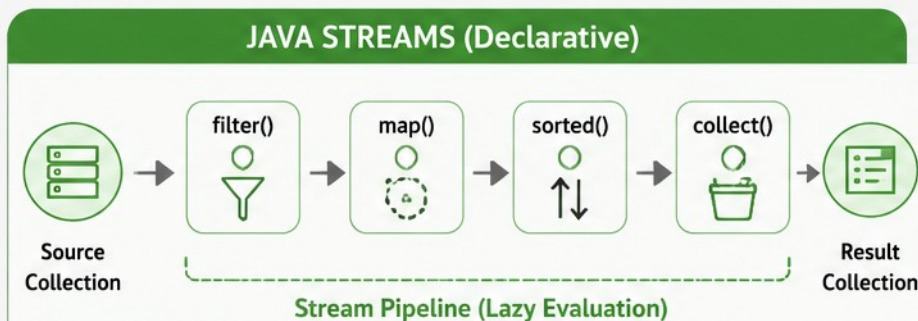
*Operations that can stop before processing the whole stream.*

| OPERATION                             | DESCRIPTION                              | EXAMPLE BEHAVIOR                                   |
|---------------------------------------|------------------------------------------|----------------------------------------------------|
| <code>findFirst() / findAny()</code>  | Returns the first / any matching element | Stops when a match is found                        |
| <code>anyMatch()</code>               | Tests whether any element matches        | Stops when a match is found                        |
| <code>allMatch() / noneMatch()</code> | Tests whether all / no elements match    | Stops on first failure / match                     |
| <code>limit(n)</code>                 | Limits the number of elements            | Stops after n elements                             |
| <code>count()</code>                  | Counts all elements                      | Processes the entire stream (not short-circuiting) |

**KEY TAKEAWAY** Use short-circuiting operations when possible — they can turn an  $O(n)$  scan into an early exit.

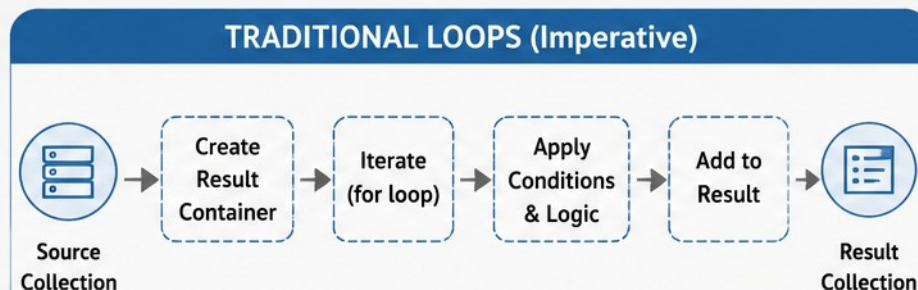
# STREAMS vs TRADITIONAL LOOPS (Java)

Two ways to process collections – same outcome, different approach



**Example:** Get names of employees earning > 50000, convert to upper case and sort

```
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(e -> e.getName().toUpperCase())
    .sorted()
    .collect(Collectors.toList());
```



**Example:** Get names of employees earning > 50000, convert to upper case and sort

```
List<String> result = new ArrayList<>();
for (Employee e : employees) {
    if (e.getSalary() > 50000) {
        result.add(e.getName().toUpperCase());
    }
}
Collections.sort(result);
```

VS

|                                                         |                      |                                                    |
|---------------------------------------------------------|----------------------|----------------------------------------------------|
| ✓ Declarative style – <b>What</b> to do, not <b>How</b> | 🎯 <b>Approach</b>    | ➖ Imperative style – Step by step <b>How</b> to do |
| ✓ Concise and readable                                  | ≡ <b>Code Size</b>   | ➖ More verbose                                     |
| ✓ Lazy evaluation – operations executed on demand       | 🕒 <b>Evaluation</b>  | ➖ Eager evaluation – executed immediately          |
| ✓ Built-in support for parallel processing              | ⚡ <b>Performance</b> | ➖ Manual effort needed for parallelism             |
| ✓ Easy to chain multiple operations                     | 🔗 <b>Composition</b> | ➖ Harder to compose, more boilerplate              |
| ✓ Less prone to errors (e.g., forgetting to add)        | 🛡️ <b>Safety</b>     | ➖ More room for errors                             |



Use **Streams** for concise, readable and functional style code.

Use **Traditional Loops** when you need fine-grained control or complex iterations.

# EXAMPLE PROBLEM

*Find HR employees earning > 60,000, sort by name, and collect into a list.*

| ID | NAME    | DEPARTMENT | SALARY |
|----|---------|------------|--------|
| 1  | Alice   | HR         | 70000  |
| 2  | Bob     | IT         | 55000  |
| 3  | Charlie | HR         | 65000  |
| 4  | David   | HR         | 75000  |
| 5  | Eve     | IT         | 60000  |
| 6  | Frank   | HR         | 50000  |

Expected Output: Alice, Charlie, David

**KEY TAKEAWAY** David qualifies too — salary 75000 in HR — even though he's last alphabetically among the three.

# SOLUTION: USING STREAMS

*Concise, expressive, and easy to convert to a parallel stream.*

```
List<String> result = employees.stream()  
    .filter(e -> e.getDepartment().equals("HR"))  
    .filter(e -> e.getSalary() > 60000)  
    .map(Employee::getName)  
    .sorted()  
    .collect(Collectors.toList()); // [Alice, Charlie, David]
```

- Why it's better: concise and expressive; focus on what, not how; easy to read and maintain; easy to parallelize.

**KEY TAKEAWAY** The stream version reads almost like a description of the requirement itself.



# SOLUTION: USING TRADITIONAL LOOPS

*Same result, more code, and manual state management.*

```
List<String> result = new ArrayList<>();
for (Employee e : employees) {
    if (e.getDepartment().equals("HR")) {
        if (e.getSalary() > 60000) {
            result.add(e.getName());
        }
    }
}
Collections.sort(result); // [Alice, Charlie, David]
```

- Considerations: more lines of code, manual state management, more prone to errors, harder to parallelize.

**KEY TAKEAWAY** Nested if statements and manual sorting are exactly the kind of boilerplate streams eliminate.

# WHEN TO USE WHAT?

*Match the tool to the shape of the problem.*



## Use Streams When

- Performing transformations, filtering, mapping, sorting, or aggregations.
- You want concise, readable code.
- You may benefit from parallel processing.
- Working with large collections or I/O data.



## Use Loops When

- You need fine-grained control over iteration.
- Working with complex business logic.
- You need early exit or custom control flow.
- Performance tuning simple iterations.

**KEY TAKEAWAY** Prefer streams for data processing; prefer loops for complex control flow.

# PERFORMANCE CONSIDERATIONS

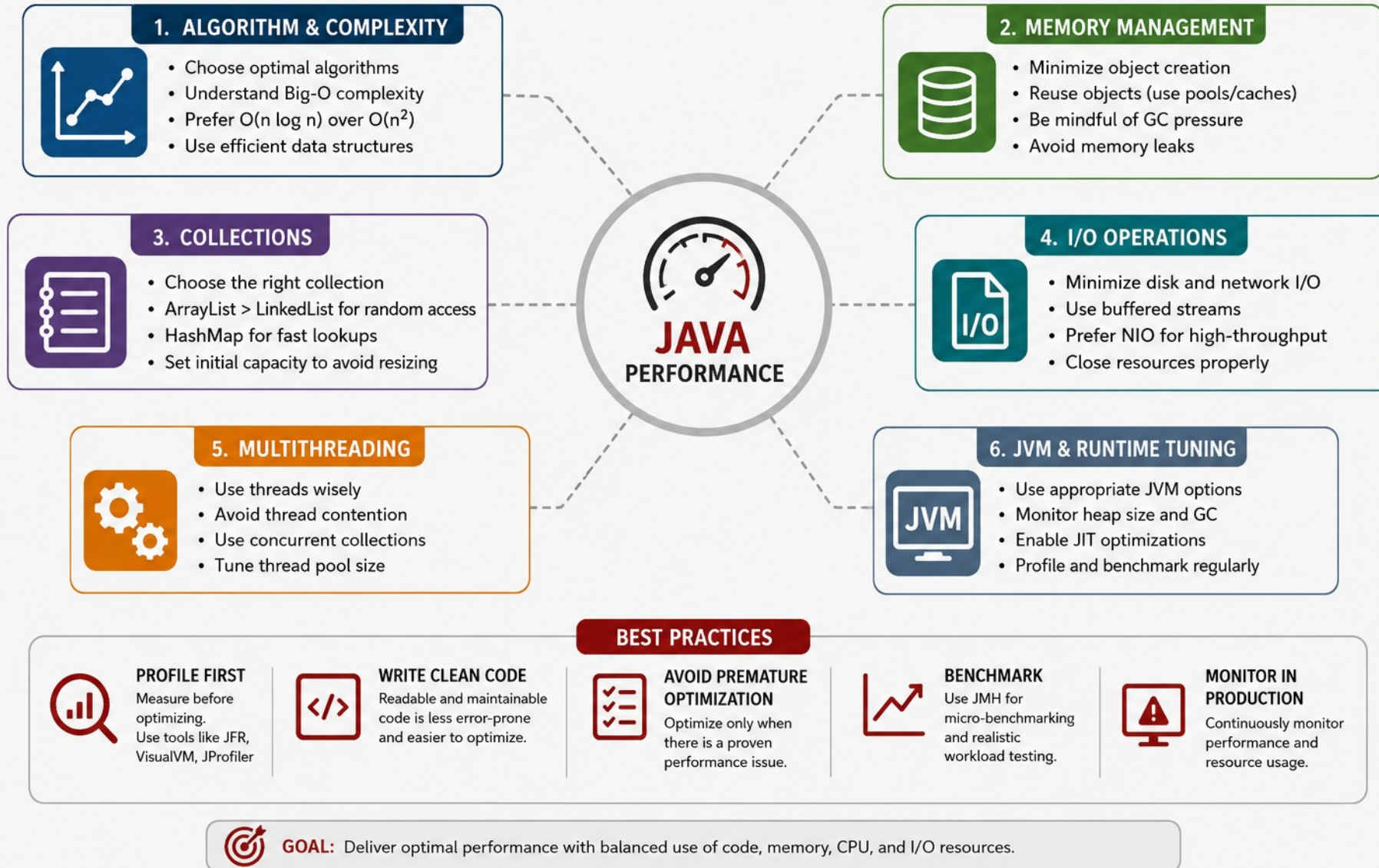
*Streams are expressive, but understanding the trade-offs helps you write efficient, scalable applications.*

- Performance depends on data size, operation type, and pipeline design.
- Use streams for readability and maintainability, but measure when performance is critical.
- Avoid unnecessary operations and boxing/unboxing.
- Be careful with stateful operations (`sorted()`, `distinct()`, `limit()`, etc.).
- Parallel streams can improve performance for large datasets, but may add overhead for small data.
- Always benchmark and profile before and after optimization.

**KEY TAKEAWAY** Streams are not always faster — measure, don't guess.

# PERFORMANCE CONSIDERATIONS IN JAVA

Key areas that impact performance and how to optimize them



# WHEN STREAMS PERFORM WELL — AND WHEN THEY DON'T

*Two lists worth memorizing before you reach for a stream by default.*



## Streams Perform Well When...

- Large datasets benefit from declarative processing.
- Complex transformations, filtering, grouping, aggregations.
- Leveraging parallel processing.
- Readability and maintainability matter.



## Streams May Not Be Ideal When...

- Very small datasets (stream creation overhead).
- Simple iteration with trivial logic.
- Performance-critical inner loops.
- Fine-grained control over iteration is required.

**KEY TAKEAWAY** The overhead of building a stream pipeline is real — it just doesn't matter until the dataset is trivially small.



# OPERATION COST GUIDE

*Not all stream operations cost the same.*

| OPERATION TYPE               | EXAMPLES                              | COST     | NOTES                                              |
|------------------------------|---------------------------------------|----------|----------------------------------------------------|
| Stateless Intermediate       | filter(), map(), peek()               | Low      | Simple and efficient, can be fused.                |
| Stateful Intermediate        | sorted(), distinct(), limit(), skip() | Med-High | May require buffering or reordering.               |
| Short-Circuiting             | anyMatch(), findFirst(), limit()      | Low-Med  | Can stop early, saves work.                        |
| Terminal (Non-Short-Circuit) | collect(), forEach(), reduce()        | Med-High | Processes all elements.                            |
| Parallel Streams             | parallelStream()                      | Variable | High overhead for small data; benefits large data. |

**KEY TAKEAWAY** Place cheap, stateless operations like filter() before expensive ones like sorted() to reduce work early.

# STREAMS VS LOOPS: A REAL EXAMPLE

*Sum of squares of even numbers, 0 to 1,000,000 — both approaches, same result.*

## USING STREAMS

```
long sum = IntStream.range(0, 1_000_000)
    .filter(i -> i % 2 == 0)
    .mapToLong(i -> (long) i * i)
    .sum();
```

## USING A TRADITIONAL LOOP

```
long sum = 0L;
for (int i = 0; i < 1_000_000; i++) {
    if (i % 2 == 0) sum += (long) i * i;
}
```

- Parallel streams — Rule of Thumb: use only for large datasets (10,000+ elements) with CPU-intensive operations.
- Costs: thread management overhead, order may not be preserved, not beneficial for I/O-bound tasks.

**KEY TAKEAWAY** Both produce the same result — performance varies with JVM optimizations, data size, and pipeline complexity.

# SAMPLE BENCHMARK & TUNING CHECKLIST

*JMH results for 1 million integers — parallel only wins at scale.*

| APPROACH            | ENVIRONMENT      | AVG TIME (MS) | NOTES                     |
|---------------------|------------------|---------------|---------------------------|
| Traditional Loop    | Single Thread    | 28.7          | Baseline                  |
| Stream (Sequential) | Single Thread    | 34.2          | ~19% slower (overhead)    |
| Stream (Parallel)   | 8 cores          | 9.1           | ~3× faster for large data |
| Stream (Parallel)   | 8 cores, 1K data | 35.8          | Overhead dominates        |

- Is the dataset large enough to justify streams or parallel streams?
- Can I use primitive streams (IntStream, LongStream) to avoid boxing?
- Are expensive operations placed after filters? Can a short-circuiting terminal operation be used?
- Have I removed unnecessary intermediate operations, and benchmarked with representative data?

**KEY TAKEAWAY** Parallel streams only pay off at scale — at 1,000 elements the overhead dominates and it's actually slower.

# BEST PRACTICES & COMMON PITFALLS

*What to do, and the mistakes that catch even experienced developers.*



## Best Practices

- Avoid unnecessary boxing/unboxing — use primitive streams.
- Place `filter()` before expensive operations.
- Use `limit()`, `findFirst()`, `anyMatch()` to short-circuit.
- Measure performance with JMH, VisualVM, JProfiler.

Keep intermediate operations stateless and side-effect free; use meaningful method references/lambdas.

Prefer `toUnmodifiableList/Set/Map()` (Java 10+) for immutable results; streams suit large or infinite sources too.



## Common Pitfalls

- Using streams for simple loops adds overhead without benefit.
- Using parallel streams for small datasets.
- Ignoring the cost of stateful operations like `sorted()`.
- Assuming streams are always faster without measuring.

Reusing an already-consumed stream throws `IllegalStateException` — call `.stream()` fresh each time.

**KEY TAKEAWAY** Understand the trade-offs, design your pipeline wisely, and measure performance to make the right choice.

# ENTERPRISE USE CASES FOR STREAMS

*Streams simplify complex data processing and help build efficient, scalable enterprise applications.*

- Ideal for collections, large datasets, and real-time data flows.
- Improve productivity with concise, expressive code.
- Easily support filtering, transformation, aggregation, grouping, and summarization.
- Enable parallel processing for performance at scale.
- Help build maintainable, testable, and robust applications.

**KEY TAKEAWAY** Streams give enterprise teams a uniform API for very different data-processing needs.



# ENTERPRISE USE CASES FOR STREAMS IN JAVA



# WHY STREAMS IN ENTERPRISE?

*Five reasons enterprise teams standardize on the Streams API.*



## Higher Productivity

Less boilerplate, more focus on business logic.



## Better Maintainability

Readable, modular, easy to refactor.



## Performance at Scale

Lazy evaluation and parallel streams improve throughput.



## Versatility

Works with collections, databases (JPA), files, real-time data.



## Consistency

Uniform API for diverse data-processing needs.

**KEY TAKEAWAY** Consistency is underrated — the same filter/map/collect vocabulary applies everywhere in the codebase.

# ENTERPRISE USE CASES — PART 1

*Filtering, transformation, aggregation, and grouping.*

| USE CASE                    | DESCRIPTION                                               | EXAMPLE OPERATIONS                    | BUSINESS VALUE                   |
|-----------------------------|-----------------------------------------------------------|---------------------------------------|----------------------------------|
| Data Filtering & Validation | Filter records based on business rules and quality checks | filter(), allMatch(), anyMatch()      | Accurate, high-quality data      |
| Data Transformation         | Transform data into required formats or structures        | map(), flatMap()                      | Standardizes data for processing |
| Aggregation & Summarization | Totals, averages, counts, min/max, summaries              | collect(), reduce(), summarizingInt() | Business insights and KPIs       |
| Grouping & Categorization   | Group data by attributes, aggregate per group             | groupingBy(), partitioningBy()        | Helps reporting and segmentation |

**KEY TAKEAWAY** Filtering and transformation are the entry point for almost every enterprise data pipeline.

# ENTERPRISE USE CASES — PART 2

*Joining, deduplication, reporting, and real-time processing.*

| USE CASE                   | DESCRIPTION                                         | EXAMPLE OPERATIONS                  | BUSINESS VALUE                    |
|----------------------------|-----------------------------------------------------|-------------------------------------|-----------------------------------|
| Data Joining & Merging     | Combine data from multiple sources or collections   | flatMap(), joining(), toMap()       | Unified views across systems      |
| Deduplication              | Remove duplicates, ensure unique records            | distinct(), toSet()                 | Data integrity, less redundancy   |
| Reporting & Dashboards     | Generate reports with aggregated data               | groupingBy(), counting(), summing() | Real-time analytics for decisions |
| Real-time Event Processing | Process streaming events from queues, logs, sensors | filter(), map(), windowing          | Low-latency insights and alerting |

**KEY TAKEAWAY** Real-time event processing is where streams and message queues (Kafka, RabbitMQ) meet in production systems.

# CODE EXAMPLE: EMPLOYEE ANALYTICS

*Average salary per department, for employees earning more than 60,000.*

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .filter(e -> e.getSalary() > 60000)
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)));
avgSalaryByDept.forEach((dept, avg) ->
    System.out.println(dept + " : " + avg));
```

| DEPT    | AVG SALARY |
|---------|------------|
| HR      | 74250.0    |
| IT      | 68500.0    |
| Finance | 91200.0    |

**KEY TAKEAWAY** groupingBy() combined with a downstream collector is the pattern behind most enterprise analytics queries.

# STREAMS IN ENTERPRISE SYSTEMS

*Where streams show up when processing real production data.*

- Databases (query processing) — Files & Logs (ETL, parsing, analytics)
- Message Queues (Kafka, RabbitMQ, ActiveMQ) — APIs & Services (transform/aggregate responses)
- Reports & Dashboards (analytics and visualizations)



## Parallel Benefits

- Utilizes multiple processor cores.
- Faster processing for large datasets.



## Considerations

- Not beneficial for small datasets.
- Order may not be preserved.
- Avoid shared mutable state.

**KEY TAKEAWAY** `parallelStream()` is best suited for large datasets and computationally intensive workloads.



# PARALLEL STREAM EXAMPLE & BEST PRACTICES

*Filtering high-value orders across many cores.*

```
List<Customer> highValueCustomers = orders.parallelStream()  
    .filter(o -> o.getAmount() > 10000)  
    .map(Order::getCustomer)  
    .distinct()  
    .collect(Collectors.toList());
```

- Prefer streams for complex data processing; use meaningful method references and lambdas.
- Avoid side effects in intermediate operations; always handle null values appropriately.
- Consider parallel streams only after profiling and testing; measure performance in production.

Use parallel streams for large datasets (10,000+ elements) with CPU-heavy work; avoid them for small collections or I/O-bound tasks, where overhead often makes them slower.

**KEY TAKEAWAY** Choose streams for clarity and productivity; choose parallel streams for scale — but only after measuring, since split/merge overhead can make them slower on small data.

# TRY IT YOURSELF — EXERCISE 8: PARALLELSTREAM BONUS

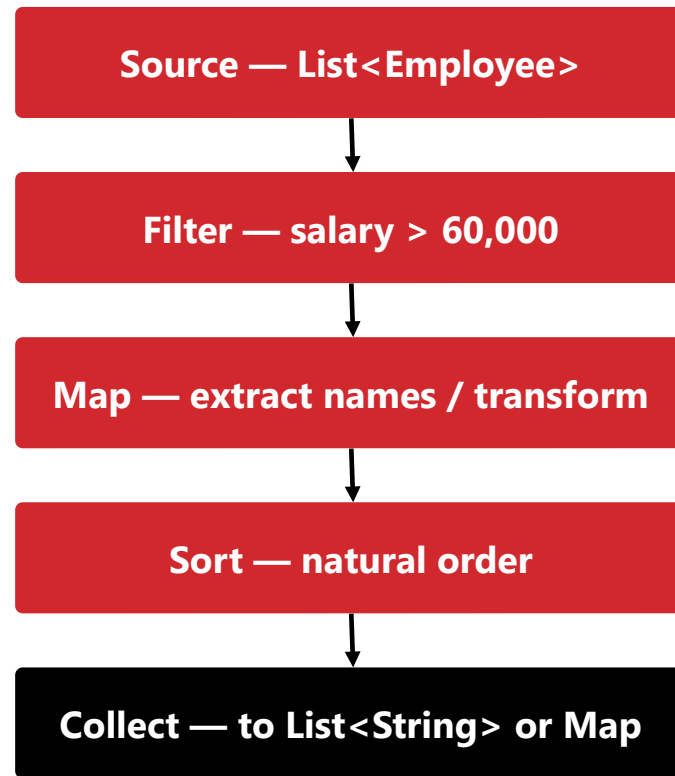
*Reinforces: the parallel stream example you just saw.*

| STEP        | CODE                                                     | RESULT                                           |
|-------------|----------------------------------------------------------|--------------------------------------------------|
| Goal        | Compare count() using stream() vs. parallelStream()      | Both counts match                                |
| Code        | .stream().count() vs .parallelStream().count()           | Same result either way                           |
| Key concept | On tiny data, parallel overhead can outweigh any benefit | Ties to the parallel stream example you just saw |
| Reference   | exercises/exercise-08-parallel-bonus.md                  | Full starter code and steps                      |

**TRY IT NOW** Complete Exercise 8 before continuing — see exercises/exercise-08-parallel-bonus.md.

# PRE-LAB EXERCISE PIPELINE SHAPE

*Filter, map, sort, collect — the same shape you will reuse throughout Lab 6.*



**KEY TAKEAWAY** Terminal operations (collect(), forEach(), count()) are what actually trigger execution.

# PRE-LAB EXERCISES: CODE STARTER (SAMPLE)

*The Employee record and dataset provided for the pre-lab exercises.*

```
record Employee(  
    int id, String name, String department, double salary) {  
  
    // record – canonical accessors: id(), name(), department(), salary()  
}  
  
List<Employee> employees = List.of(  
    new Employee(1, "Alice", "HR", 72_000),  
    new Employee(2, "Bob", "IT", 65_000),  
    new Employee(3, "Charlie", "HR", 80_000),  
    new Employee(4, "Diana", "Finance", 90_000),  
    new Employee(5, "Evan", "IT", 55_000)  
);
```

**KEY TAKEAWAY** This exact 5-employee dataset drives every expected output in the pre-lab exercises — Lab 6 later uses a larger 25-employee roster.

# PRE-LAB EXERCISES: EXPECTED OUTPUT (SAMPLE)

*What each of the seven exercises should produce, given the starter dataset.*

1. Anonymous: true    Lambda: true
2. [Alice, Bob, Charlie, Diana]
3. [Alice, Bob, Charlie, Diana, Evan]
4. Highest Salary: 90000.0 (Diana)    Lowest Salary: 55000.0 (Evan)
5. [79200.0, 71500.0, 88000.0, 99000.0, 60500.0]
6. {HR=2, IT=2, Finance=1}
7. [Alice, Charlie]

*Useful operations: filter(), map(), sorted(), collect(), groupingBy(), max()/min(), Comparator.comparing()*

**KEY TAKEAWAY** Actual output may vary if the dataset changes — but the operations you'd use stay the same.

# LAB 6 OVERVIEW — EMPLOYEE ANALYTICS SYSTEM

*Build a menu-driven console app that turns a 25-employee roster into management reports using Streams and Lambdas.*



## Lab 6 Objectives

- Use lambdas and core functional interfaces (Predicate, Function, Consumer, Supplier).
- Build stream pipelines: filter, map, sorted, distinct, limit, skip.
- Aggregate with reduce(), collectors, groupingBy(), and partitioningBy().
- Handle missing values safely with Optional and build an analytics Dashboard.



## Prerequisites

- JDK 21 and IntelliJ IDEA Community (or VS Code).
- Lab 0 environment setup completed.
- Comfort with List, packages, and Lab 5 service-layer patterns.
- Module 6 pre-lab exercises completed (see previous slides).

**KEY TAKEAWAY** Lab 6 is the real hands-on capstone for Module 6 — a full Employee Analytics System, not just the pre-lab exercises.



# LAB 6 DATASET — EMPLOYEE ANALYTICS

*The Employee model used throughout Lab 6 — 25 sample employees across five departments.*

| FIELD      | TYPE    | DESCRIPTION                       | EXAMPLE    |
|------------|---------|-----------------------------------|------------|
| employeeId | String  | Primary employee key              | E001       |
| name       | String  | Employee name                     | John Smith |
| department | String  | IT, HR, Finance, Sales, Marketing | IT         |
| salary     | double  | Annual salary (48,000–165,000)    | 165000     |
| active     | boolean | Employment status                 | true       |

**KEY TAKEAWAY** Employee also has experience and rating fields — seven in total feed the Dashboard and every report.

# LAB 6 CORE TASKS — MENU-DRIVEN CONSOLE

*Nine graded menu options (1–9) — build them in order before any bonus features.*

- 1. Display Employees — print the full 25-employee roster via `forEach()`.
- 2. Employees By Department — group with `Collectors.groupingBy(Employee::getDepartment)`.
- 3. Salary Report — `reduce()`, `summarizingDouble()`, and `partitioningBy()` by salary threshold.
- 4. Top Performers — filter and sort employees with rating  $\geq 4$ .
- 5. Highest Salary — `max(Comparator.comparingDouble(...))` returned as `Optional<Employee>`.
- 6. Department Statistics — `groupingBy()` + `summarizingDouble()` per department.
- 7. Active Employees — `filter(Employee::isActive)`.
- 8. Dashboard — headcount, salary stats, top performer, top 5 salaries, active/inactive counts.
- 9. Exit — clean shutdown of the Scanner and menu loop.

**KEY TAKEAWAY** Complete CORE menu options 1–9 first; bonus features (menu 10+) come only after CORE works.

# LAB 6 — KEY STREAM OPERATIONS USED

*Five categories of operations power every report in the Employee Analytics System.*

| CATEGORY              | OPERATION                                              | EXAMPLE                                                                        |
|-----------------------|--------------------------------------------------------|--------------------------------------------------------------------------------|
| Functional Interfaces | Predicate, Function, Consumer, Supplier                | Predicate<Employee> highEarner = e -> e.getSalary() > 100_000                  |
| Filtering & Mapping   | filter(), map(), method references                     | filter(Employee::isActive).map(Employee::getName)                              |
| Sorting & Slicing     | sorted(), distinct(), limit(), skip()                  | sorted(Comparator.comparingDouble(Employee::getSalary).reversed()).limit(5)    |
| Reduction             | reduce(),<br>mapToDouble().sum()/average()             | mapToDouble(Employee::getSalary).average()                                     |
| Collectors & Optional | groupingBy(), partitioningBy(),<br>summarizingDouble() | groupingBy(Employee::getDepartment,<br>summarizingDouble(Employee::getSalary)) |

**KEY TAKEAWAY** Every menu option composes these same five categories — that judgment, not the syntax, is what transfers to CRM reporting later.

# LAB 6 — SUBMISSION CHECKLIST

*Everything to submit and double-check once CORE menu options 1–9 and the Dashboard are working.*

- 1. Complete Java Project — Employee, EmployeeData, EmployeeService, ReportService, Main under com.academy.analytics.
  - 2. Screenshots — the running menu, filters/groups, and the Dashboard (option 8).
  - 3. Stream Operations Table — which operations you implemented and where.
  - 4. README / LMS Notes — overview, operations used, functional interfaces, how to compile and run.
  - 5. Reflection Answers — written in notes/lab6-answers.md.
  - 6. Bonus (Optional) — labeled stretch challenges from menu options 10+.
7. Avoid Common Pitfalls — don't reuse a consumed stream; never call `.get()` on `Optional` (use `orElse()/ifPresentOrElse()`); sort before both `limit()` and `skip()`; recompile with `javac -d out` after every change.
8. Reflection Topic — in notes/lab6-answers.md, explain why streams postpone work until a terminal operation, and how `groupingBy()` will reuse for CRM reporting later.

**KEY TAKEAWAY** Graders recompile your sources with `javac -d out` — the Dashboard screenshot is the single most important piece of evidence.

# LAB 6 — EVALUATION RUBRIC & DASHBOARD OUTPUT

*How the 100-mark rubric is graded, and what the Dashboard (menu option 8) should print.*

- Graded out of 100: Project Structure (10), Lambda Expressions (10), Functional Interfaces (10), Stream Pipeline (15).
- Filtering/Mapping/Sorting (15), Collectors & Grouping (15), Reduction & Statistics (10), Dashboard & Reports (10), Code Quality (5).

```
Employees : 25   Average Salary : 100680   Highest Salary : 165000
Departments : 5   Top Performer : John Smith (Rating 5)
Highest Paid Department : IT
Active Employees : 23   Inactive Employees : 2
```

**KEY TAKEAWAY** Stream Pipeline, Filtering/Mapping/Sorting, and Collectors & Grouping carry the most marks — 45 of the 100.

# MODULE SUMMARY AND NEXT STEPS

*Java Streams and Lambda expressions provide a powerful, declarative, and concise way to process data.*

- Streams enable functional-style programming using lambda expressions.
- Pipelines are built with intermediate operations and executed by terminal operations.
- Operations are lazy, concise, and promote readability.
- Streams support powerful data processing on collections, arrays, files, and more.
- Use Streams for maintainable, testable, and scalable enterprise applications.

Benefits compound across a codebase: concise & expressive, readable & maintainable, flexible across data sources, and performant at scale via parallel processing.

You can now build filter/map/collect pipelines, use Collectors for reports, and choose sequential vs parallel streams appropriately.

Did you know? The Streams API launched in Java 8. Popular entry points: `collection.stream()`, `Arrays.stream(array)`, `Stream.of(values...)`.

**KEY TAKEAWAY** This module covered the core concepts, operations, and practical usage of Streams through hands-on labs.



# CORE CONCEPTS COVERED

*Six pillars this module built up, one at a time.*

## **Lambda Expressions**

- Provide concise, anonymous functions.

## **Stream Pipelines**

- Source → Intermediate Operations → Terminal Operation.

## **Lazy Evaluation**

- Intermediate operations execute only when needed.

## **Stream Operations**

- Filtering, mapping, sorting, aggregation, collecting.

## **Collectors Framework**

- Flexible ways to collect and summarize results.

## **Performance**

- When streams help, and when loops are more appropriate.

**KEY TAKEAWAY** Every later topic in this module built directly on lambda expressions and the pipeline shape.

# WHEN TO USE — AND AVOID — STREAMS

*The decision comes down to data size, complexity, and control needs.*



## Use Streams When You Need...

- Complex data transformations.
- Filtering, mapping, and aggregation.
- Processing large datasets.
- Parallel processing for performance.
- Improved readability and maintainability.



## Consider Loops When...

- Very simple iterations (stream overhead unnecessary).
- Performance-critical tight loops.
- Small datasets where readability gains are minimal.

**KEY TAKEAWAY** Loops still have their place when you need full control and flexibility.

# IMPORTANT OPERATIONS REFERENCE

*Eight operation categories, and what each one does.*

| CATEGORY    | COMMON OPERATIONS                   | PURPOSE                              |
|-------------|-------------------------------------|--------------------------------------|
| Filtering   | filter()                            | Select elements matching a condition |
| Mapping     | map()                               | Transform each element               |
| Sorting     | sorted()                            | Sort stream elements                 |
| Aggregation | collect(), reduce(), summarizing*() | Aggregate or summarize data          |
| Grouping    | groupingBy()                        | Group elements by key                |
| Counting    | count()                             | Count elements                       |
| Finding     | findFirst(), anyMatch(), allMatch() | Search or match elements             |
| Collecting  | toList(), toSet(), toMap()          | Collect into data structures         |

**KEY TAKEAWAY** Bookmark this table — it's the fastest lookup for choosing the right operation under time pressure.

# KNOWLEDGE CHECK — MULTIPLE CHOICE (1–6)

*Choose the best answer. Correct answers are marked ✓.*

**1. Which of the following best describes a Stream?**

- A) A data structure that stores elements   **B) A sequence of elements supporting functional-style operations** ✓   C) A class used to read data from files   D) A thread used for parallel processing

**2. Which operation type does not trigger execution of the stream pipeline?**

- A) filter()   B) map()   **C) sorted()** ✓   D) collect()

**3. What is the purpose of the map() operation?**

- A) To filter elements based on a condition   **B) To transform each element** ✓   C) To sort elements   D) To remove duplicates

**4. Which terminal operation returns the number of elements in a stream?**

- A) count()** ✓   B) forEach()   C) collect()   D) reduce()

**5. How does lazy evaluation benefit stream pipelines?**

- A) It improves code readability   **B) It avoids unnecessary computation** ✓   C) It increases memory usage   D) It executes operations in parallel

**6. Which method processes each element without returning a result?**

- A) map()   B) filter()   **C) forEach()** ✓   D) reduce()

**KEY TAKEAWAY** Six more multiple-choice questions continue on the next slide.

# KNOWLEDGE CHECK — MULTIPLE CHOICE (7–12)

*Choose the best answer. Correct answers are marked ✓.*

**7. Which collector would you use to convert a stream to a List?**

A) `Collectors.toSet()`   **B) `Collectors.toList()`** ✓   C) `Collectors.toMap()`   D) `Collectors.toCollection()`

**8. What is the correct syntax to create a stream from a `List<Employee>` named `emps`?**

A) `Stream<Employee> s = new Stream<>(emps);`   **B) `Stream<Employee> s = emps.stream();`** ✓   C) `Stream<Employee> s = Stream.of(emps);`   D) `Stream<Employee> s = emps.toString();`

**9. Which operation is used to sort elements in natural order?**

**A) `sorted()`** ✓   B) `sort()`   C) `order()`   D) `arrange()`

**10. Which of the following is true about parallel streams?**

A) They always improve performance.   B) They maintain encounter order by default.   **C) They are created using `parallelStream()`.** ✓   D) They can be used only with primitive types.

**11. Which operation combines the elements of a stream into a single value?**

A) `collect()`   **B) `reduce()`** ✓   C) `map()`   D) `filter()`

**12. Which statement is true about Lambda expressions?**

A) They can modify only instance variables.   B) They must have a return statement.   **C) They must capture only final or effectively final variables.** ✓   D) They can throw checked exceptions only.

**KEY TAKEAWAY** 12 questions, testing the full span of this module — from stream basics to lambda variable capture.

# SELF-ASSESSMENT & BONUS CHALLENGE

*How confident are you with Module 6 concepts?*



## Confident

Ready to apply these concepts independently.



## Average

Comfortable with the basics, need more practice.



## Need Review

Revisit the pipeline and lazy evaluation sections.

## BONUS CHALLENGE

Using `List<Employee> employees`: filter salary > 60,000, sort by salary descending, get the top 3, collect their names into a `List<String>`, and print the result.

*Hint: `filter()` → `sorted()` → `limit()` → `map()` → `collect()`*

**KEY TAKEAWAY** If you can chain `filter` → `sorted` → `limit` → `map` → `collect` from memory, you've mastered this module.

# Nicely done

You can write concise, declarative pipelines with Streams and Lambdas instead of imperative loops.

Exception handling and robust error management are next in your toolkit.